

ROADMAPBENCH: Evaluating Long-Horizon Agentic Software Development Across Version Upgrades

Xinbo Xu^{1,2}, Ruihan Yang³, Haiyang Shen^{1,2}, Wendong Xu^{1,4}, Bofei Gao², Ruoyu Wu^{1,2}, Kean Shi^{1,2}, Weichu Xie², Xuanzhong Chen^{1,5}, Ming Wu⁶, Jason Zeng⁶, Michael Heinrich⁶, Elvis Zhang⁷, Liang Chen^{1†}, Kuan Li^{1†}, Baobao Chang^{2†}

¹UniPat AI ²Peking University ³Fudan University

⁴The University of Hong Kong ⁵Tsinghua University ⁶0G Labs ⁷Pipeline Lab

Abstract

Coding agents are increasingly deployed in real software development, where a single version iteration requires months of coordinated work across many files. However, most existing benchmarks focus predominantly on single-issue bug fixes from Python repositories, with coarse pass/fail evaluation outcomes, and thus fail to capture long-horizon, multi-target development at real engineering scale. To address this gap, we present ROADMAPBENCH, a benchmark of 115 long-horizon coding tasks grounded in real open-source version upgrades across 17 repositories and 5 programming languages. Each task places the agent on a source-version code snapshot and provides a multi-target roadmap instruction requiring it to implement the functionality introduced in the target version, with a median modification of 3,700 lines across 51 files. We conduct a systematic evaluation on thirteen frontier models and find that even the strongest, Claude-Opus-4.7, resolves only 39.1% of tasks, while the weakest achieves merely 5.2%, in stark contrast to existing bug-fix benchmarks, suggesting that long-horizon software development remains a largely unsolved problem.

 **Code:** <https://github.com/UniPat-AI/RoadmapBench>

 **Dataset:** <https://huggingface.co/datasets/UnipatAI/RoadmapBench>

 **Leaderboard:** <https://unipat.ai/benchmarks/RoadmapBench>

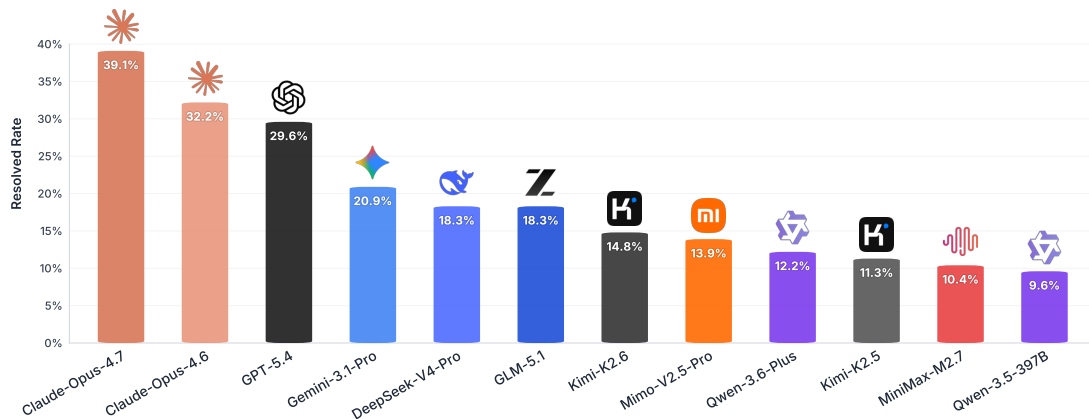


Figure 1: **RoadmapBench Leaderboard.** Resolved rate of top-performing models evaluated with OpenHands across 115 multi-target software evolution tasks spanning 5 languages and 17 repositories. Even the best-performing model resolves only 39.1% of tasks.

[†]Corresponding authors: liangchen@unipat.ai, kuanli@unipat.ai, chbb@pku.edu.cn

Table 1: Comparison with related coding benchmarks. Scope: task granularity. Subtask Score: target-level completion scoring. Solution: oracle patch size (LoC).

Benchmark	#Tasks	Lang.	Scope	Subtask Score	Solution
SWE-bench Verified OpenAI (2024)	500	Python	Commit	✗	~33 LOC
SWE-bench Pro Deng et al. (2025)	1,865	Multi	Commit	✗	~107 LOC
FeatureBench Zhou et al. (2025)	200	Python	Commit	✗	~790 LOC
TerminalBench Merrill et al. (2026)	89	Multi	Task	✗	~280 LOC
SWE-EVO Thai et al. (2025)	48	Python	Version	✗	~611 LOC
NL2Repo Ding et al. (2025)	104	Python	Repo	✗	~3,000 LOC
ROADMAPBENCH	115	Multi	Version	✓ (avg. 5 targets)	~3,700 LOC

1 Introduction

The rapid progress of large language models (LLMs) ([Anthropic, 2026a](#); [OpenAI, 2026](#); [Google DeepMind, 2025](#)) has enabled a new generation of coding agents that can plan, edit, execute, and validate software in interactive development environments ([Yang et al., 2024](#); [Zhang et al., 2024](#); [Huang et al., 2025](#); [Wang et al., 2025](#)). As these agents move beyond isolated code generation and bug fixing, the central challenge increasingly lies in sustained, multi-target software development. Evaluation is therefore shifting from **short-horizon** defect repair to **long-horizon** feature implementation. This raises a critical question: *how to evaluate an agent on multi-target, human-scale development work spanning weeks or months?*

Existing benchmarks have not kept pace with this shift (Table 1). Most current benchmarks remain short-horizon: SWE-bench ([Jimenez et al., 2024](#)) and SWE-bench Pro ([Deng et al., 2025](#)) evaluate isolated software engineering problems, with oracle solutions of ~33 and ~107 lines respectively, one to two orders of magnitude below the scale of real engineering work. Long-horizon attempts remain scarce and collapse each task to a single binary outcome, overlooking the multi-target structure that real version upgrades naturally exhibit, where developers coordinate multiple substantial changes within a single release cycle. Beyond scope and granularity, existing benchmarks remain concentrated in a limited set of heavily reused Python repositories ([Liu et al., 2023](#); [Du et al., 2023](#)), compounding contamination risk as popular codebases become more likely to appear in pre-training corpora.

To tackle these challenges, we propose ROADMAPBENCH, a benchmark of 115 long-horizon coding tasks grounded in real open-source version upgrades. Each task starts from a repository snapshot pinned to an earlier release and requires the agent to implement the behaviors introduced in the next release, with a median oracle modification of approximately 3,700 lines across multiple files and modules. We convert each upgrade into a multi-target roadmap with a median of 5 subtasks, specifying what to implement, including API signatures, parameter semantics, default values, and exception behavior, while withholding implementation details. Each subtask is verified by its own test suite and contributes to a weighted overall score, so partial progress is captured as a continuous value rather than a binary outcome. To broaden coverage, we curate 17 repositories across 5 programming languages, spanning data processing, web frameworks, ORMs, serialization, GUI toolkits, and developer tooling, with no overlap with existing benchmarks. To ensure that failures reflect genuine capability gaps rather than benchmark artifacts, we combine static validation with attribution-driven rollout-based quality control to separate task-side defects from model-side limitations and iteratively repair confirmed task issues.

We evaluate thirteen frontier models on ROADMAPBENCH and observe that no model comes close to solving the benchmark. As shown in Figure 1, even the strongest model, Claude-Opus-4.7, resolves only 39.1% of tasks, while the weakest achieves merely 5.2%. By comparison, these systems attain 80%+ scores on SWE-bench Verified ([OpenAI, 2024](#)). The Completion Score reveals a consistent pattern: models routinely complete several subtasks before stalling at integration boundaries, offering cleaner separation across capability tiers than binary outcomes alone.

In summary, our key contributions are as follows:

- We construct ROADMAPBENCH, a benchmark of 115 real open-source version-upgrade tasks across 17 repositories and 5 programming languages, establishing long-horizon multi-target software development as a distinct evaluation setting.
- We develop a construction pipeline that transforms real version upgrades into multi-target tasks, and combines static validation with rollout-based quality control to separate task-side defects from genuine model limitations and iteratively repair confirmed issues.
- We evaluate thirteen frontier models and find that resolved rates range from 5.2% to 39.1%, well below performance on existing bug-fix benchmarks, while Completion Score reveals fine-grained capability differences across domains and difficulty tiers beyond binary resolved metrics.

2 Related Work

Coding Agents. LLM-based coding agents have evolved from single-turn code generation systems to interactive software engineering agents operating in realistic development environments (Sapkota et al., 2025; Dong et al., 2025; Starace et al., 2025). OpenHands Wang et al. (2024) provides an open platform for building generalist software development agents, while Terminus 2 Merrill et al. (2026) serves as the reference agent implementation within the Harbor framework for autonomous evaluation in sandboxed environments. Commercial systems such as Claude Code Anthropic (2025) have further brought agentic coding into mainstream software development workflows. As these systems become increasingly capable, there is a growing need for benchmarks that better reflect the complexity of real-world software engineering.

Coding Benchmarks for Agents. Coding benchmarks for LLM agents have progressively evolved from function-level synthesis to more realistic software engineering tasks. HumanEval Chen et al. (2021) and MBPP Austin et al. (2021) focus on function-level code generation. The SWE-bench family Jimenez et al. (2024); OpenAI (2024); Deng et al. (2025) extends evaluation to issue resolution and long-horizon engineering tasks in real-world repositories. Later benchmarks broaden evaluation to feature-oriented development and system-level interaction, including FeatureBench Zhou et al. (2026) and TerminalBench Merrill et al. (2026). More recent work explores increasingly open-ended and long-horizon software engineering settings. NL2Repo Ding et al. (2025) evaluates full repository generation from natural language specifications without requiring agents to evolve existing large-scale codebases, while SWE-EVO Thai et al. (2025) studies Python version evolution but derives problem statements directly from release notes without explicit instruction-test alignment validation. Existing benchmarks still primarily evaluate isolated tasks rather than structured long-horizon multi-target software development processes. ROADMAPBENCH covers 17 repositories across 5 programming languages, where each instance contains around five structured subtasks together with dedicated instruction-test alignment validation. Table 1 summarizes the key differences among existing benchmarks.

3 RoadmapBench

We describe ROADMAPBENCH across three aspects: the task definition and evaluation protocol (Section 3.1), dataset statistics (Section 3.2), and the construction pipeline (Section 3.3).

3.1 Task Definition

As illustrated in Figure 2, each ROADMAPBENCH task asks an agent to implement the functionality introduced in a real version upgrade. The agent operates in a Docker environment with the repository pinned at the source version. It is given a multi-target roadmap instruction specifying *what* to implement: each target corresponds to a distinct unit of new functionality and describes the expected behavioral

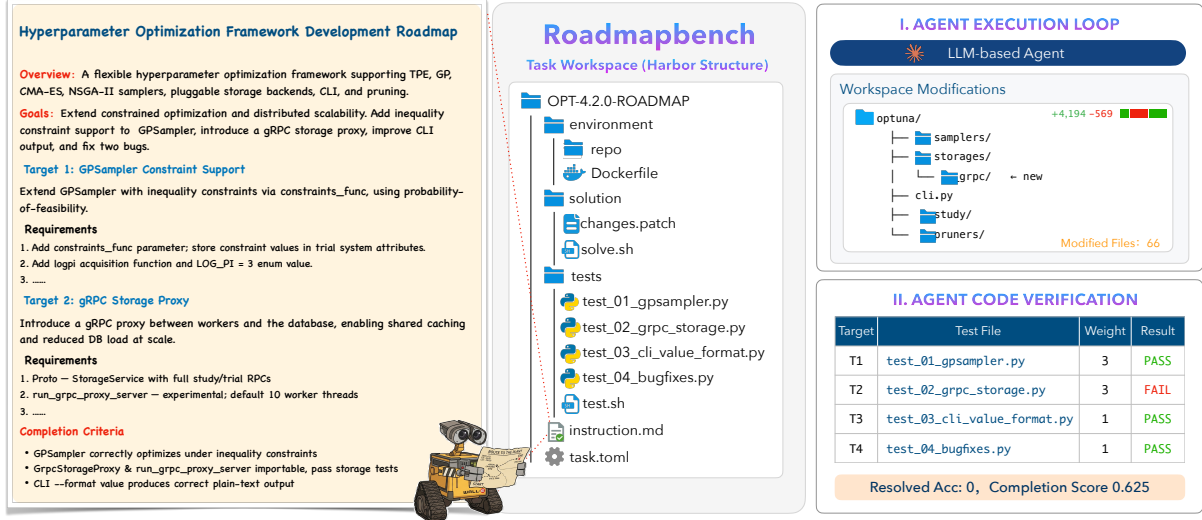


Figure 2: **Overview of a ROADMAPBENCH task.** The agent receives a source-version repository snapshot and a roadmap-style instruction, then implements the specified functionality inside a pinned Docker environment. Evaluation is performed via weighted subtask-level tests against behaviors introduced in the target version.

requirements. As in real version upgrades, where multiple substantial changes are coordinated within a single release, the targets collectively capture a unified development objective. We evaluate each task along two dimensions. A task is *resolved* if the agent passes all subtasks, providing a binary measure of complete success. To capture partial progress, we additionally compute a weighted reward: each subtask carries a weight reflecting its implementation complexity, and the reward is the weighted fraction of passed subtasks.

3.2 Dataset Statistics

The current release contains 115 tasks spanning 17 open-source repositories across five programming languages (see Appendix E for details). Oracle patches range from under 300 to over 30,000 lines changed, with a median of approximately 3,700 lines and 51 files touched. Subtask counts range from 3 to 12 with a median of 5, confirming that tasks require sustained multi-target engineering rather than single-function edits. Figure 3 shows the task distribution and oracle patch size across repositories.

3.3 Data Construction Pipeline

The pipeline proceeds in four stages (Figure 4): repository mining, task construction, static validation, and rollout-based quality control.

Stage 1: Repository Mining. We aggregate repositories from community-curated open-source project lists across five languages and apply a three-stage filter: (1) a rule-based filter retains repositories with at least 1,000 stars, five or more tagged releases, and continued release activity through 2025; (2) an in-depth search identifies repositories that maintain high-quality release documentation (see examples in Appendix G.1); (3) expert review verifies documentation quality and selects consecutive version pairs with sufficient code changes and feature narratives for task construction. This process yields 17 repositories and 115 version pairs across five languages.

Stage 2: Task Construction. Each task is built in a Docker environment pinned to the source version. The git history is preserved but all branches and tags beyond the source release are pruned, preventing the agent from inspecting target-version code through version control. We align source-to-target code

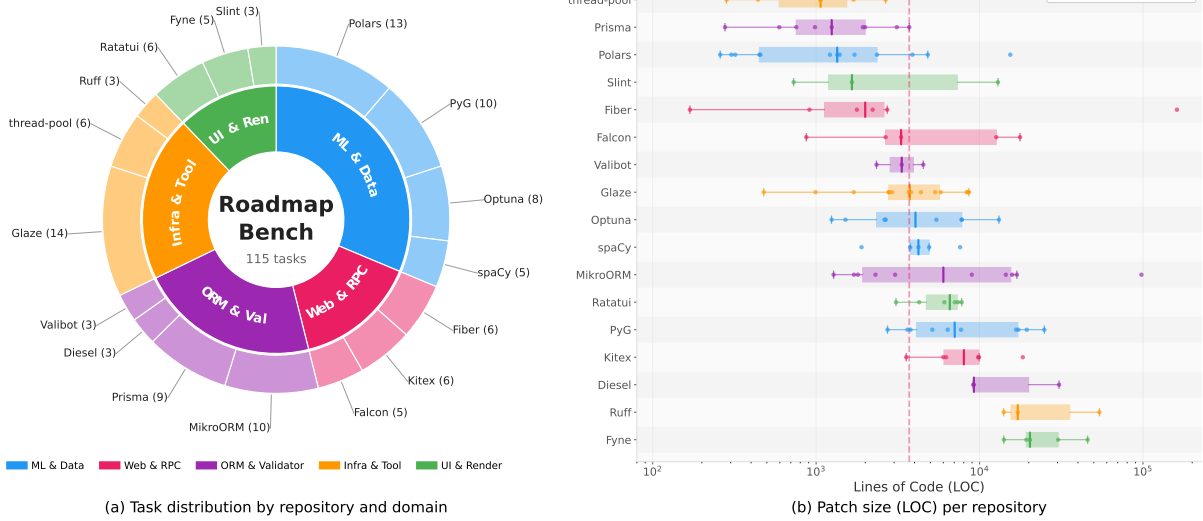


Figure 3: **Dataset overview of ROADMAPBENCH.** (a) Task count per repository (outer ring) grouped by domain (inner ring): ML & Data (36), Web & RPC (17), ORM & Val (25), Infra & Tool (23), UI & Ren (14). (b) Distribution of ground-truth patch size (lines changed) per repository, where the dashed line marks the overall median of 3,714 LOC.

diffs with release narratives to identify externally visible behavioral changes and create a multi-target roadmap instruction (`instruction.md`) specifying what to implement without revealing how. Tests are adapted from upstream suites to preserve behavioral coverage, and a gold patch is extracted from the code diff, refined against the task environment, and validated until it passes the adapted tests.

Stage 3: Static Validation. Each task is statically checked along two dimensions: *compliance*, verifying specification self-containedness, source traceability, and test validity; and *target-level correctness*, ensuring that every tested behavior for each target is specified and no test relies on unstated assumptions (details in Appendix G.2). Confirmed issues are repaired; the oracle patch is then re-run to ensure the fail-to-pass guarantee remains valid.

Stage 4: Rollout-based quality control. Agents from three capability tiers attempt each task; failures are attributed to either task-side defects (missing/ambiguous specifications) or genuine model limitations (incorrect design, buggy implementation). Task-side defects are iteratively repaired and revalidated until cleared. A task is finalized only when it contains no task-side errors, the oracle achieves full reward, and models of different tiers produce distinguishable scores (see Appendix G.3).

4 Experiments

4.1 Evaluation Setup

Models. We evaluate thirteen frontier models: Claude-Opus-4.7 [Anthropic \(2026b\)](#), Claude-Opus-4.6 [Anthropic \(2026a\)](#), GPT-5.4 [OpenAI \(2026\)](#), Gemini-3.1-Pro [Google DeepMind \(2026\)](#), DeepSeek-V4-Pro [DeepSeek-AI \(2026\)](#), GLM-5.1 [GLM-5-Team \(2026\)](#), Kimi-K2.6 [MiniMax \(2026a\)](#), Mimo-V2.5-Pro [XiaoMi \(2026\)](#), Qwen3.6-Plus [Qwen Team \(2026a\)](#), Kimi-K2.5 [Kimi Team \(2026\)](#), MiniMax-M2.7 [MiniMax \(2026b\)](#), Qwen3.5-397B [Qwen Team \(2026b\)](#), and Seed-2.0-Pro [ByteDance Seed Team \(2026\)](#). These models span multiple commercial API providers and cover a wide range of current capability tiers.

Agent scaffold. All tasks are packaged as Harbor [Harbor Framework Team \(2026\)](#) environments and can be evaluated with any Harbor-compatible agent. We use OpenHands [Wang et al. \(2024\)](#) as the

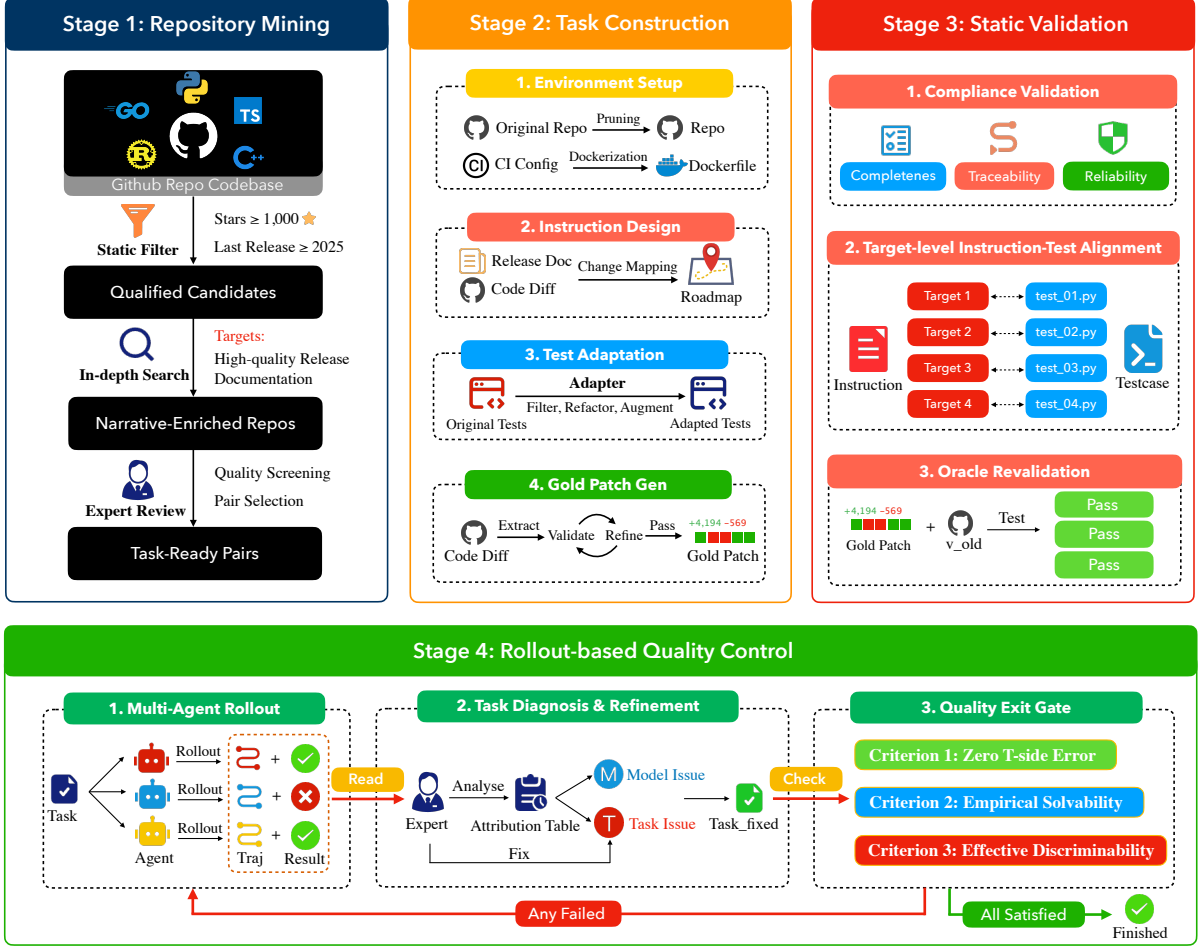


Figure 4: **ROADMAPBENCH construction pipeline**. Repository mining selects task-ready version pairs; task construction aligns release narratives with code diffs to create instructions and tests. Static validation and rollout-based quality control repair task-side defects before benchmark inclusion.

primary scaffold for all thirteen models. Each rollout runs inside a pinned Docker environment rooted at the source version. The agent may inspect and modify the repository but has no access to target-version code, test files, or the oracle patch. Future branches and upstream repository access are blocked to prevent information leakage. As an ablation, we additionally evaluate a subset of models under Terminus 2, the reference agent implementation of Harbor. Terminus 2 is designed as a neutral testing platform that runs fully autonomously in sandboxed environments, making it well suited for measuring scaffold sensitivity independent of any production-oriented design choices in OpenHands.

Inference configuration. Each task is allocated a 2-hour wall-clock budget. All models are evaluated with extended thinking enabled. For models that support configurable reasoning depth, we set reasoning effort to high for GPT-5.4, Gemini-3.1-Pro, DeepSeek-V4-Pro, and Seed-2.0-Pro, and xhigh for Claude-Opus-4.7; the remaining models use their default thinking mode.

Metrics. For task t with K_t subtasks, each subtask k carries a weight $w_{t,k}$ reflecting its relative complexity and yields a binary pass/fail result $r_{t,k} \in \{0, 1\}$. We define the per-task weighted reward as

$$s_t = \frac{\sum_{k=1}^{K_t} w_{t,k} \cdot r_{t,k}}{\sum_{k=1}^{K_t} w_{t,k}},$$

We report two primary metrics over N tasks: one for full task completion and one for partial progress. *Resolved rate* is the fraction of fully completed tasks: $RR = \frac{1}{N} \sum_t \mathbf{1}[s_t = 1]$. *Completion Score* averages s_t

Table 2: Main results on ROADMAPBENCH across 115 tasks, using a single trial per model. Domain columns report resolved rates (%). Task counts are ML & Data (36), Web & RPC (17), ORM & Val. (25), Infra. & Tool. (23), and UI & Ren. (14). **Bold** and underline denote the best and second-best domain-level results within each scaffold.

Model	Overall				Resolved Rate by Domain (%)				
	Resolved (%)	Completion Score	Avg. Turns	Output Tok. (K)	ML & Data	Web & RPC	ORM & Val.	Infra. & Tool.	UI & Ren.
OPENHANDS									
Claude-Opus-4.7	39.1	0.692	140.2	44	30.6	41.2	32.0	43.5	64.3
Claude-Opus-4.6	<u>32.2</u>	<u>0.627</u>	140.7	42	25.0	<u>29.4</u>	32.0	30.4	<u>57.1</u>
GPT-5.4	29.6	0.497	170.7	93	<u>27.8</u>	17.6	20.0	<u>39.1</u>	50.0
Gemini-3.1-Pro	20.9	0.439	133.4	26	8.3	23.5	24.0	26.1	35.7
DeepSeek-V4-Pro	18.3	0.486	140.2	64	8.3	17.6	24.0	17.4	35.7
GLM-5.1	18.3	0.453	163.2	38	8.3	11.8	<u>28.0</u>	26.1	21.4
Kimi-K2.6	14.8	0.432	158.9	76	5.6	5.9	20.0	21.7	28.6
Mimo-V2.5-Pro	13.9	0.440	155.5	66	8.3	11.8	12.0	21.7	21.4
Qwen3.6-Plus	12.2	0.424	150.3	47	5.6	5.9	12.0	21.7	21.4
Kimi-K2.5	11.3	0.378	110.3	29	0.0	5.9	12.0	17.4	35.7
MiniMax-M2.7	10.4	0.332	123.5	38	5.6	0.0	8.0	26.1	14.3
Qwen3.5-397B	9.6	0.383	110.5	35	0.0	11.8	12.0	13.0	21.4
Seed-2.0-Pro	5.2	0.177	40.1	9	0.0	5.9	8.0	4.3	14.3
TERMINUS 2									
Claude-Opus-4.7	38.3	0.681	59.2	22	27.8	41.2	36.0	43.5	57.1
Claude-Opus-4.6	<u>31.3</u>	<u>0.666</u>	82.7	43	<u>19.4</u>	<u>23.5</u>	36.0	<u>39.1</u>	<u>50.0</u>
GLM-5.1	20.9	0.512	93.8	57	11.1	11.8	<u>32.0</u>	21.7	35.7
Qwen3.6-Plus	16.5	0.508	129.6	64	8.3	11.8	16.0	26.1	28.6
Kimi-K2.6	15.7	0.409	111.7	53	5.6	11.8	28.0	17.4	21.4
DeepSeek-V4-Pro	10.4	0.395	149.2	80	2.8	5.9	12.0	21.7	14.3
Mimo-V2.5-Pro	10.4	0.344	113.7	155	2.8	17.6	8.0	13.0	21.4
Qwen3.5-397B	10.4	0.337	90.1	43	2.8	5.9	12.0	21.7	14.3
Kimi-K2.5	7.8	0.360	90.2	33	0.0	0.0	16.0	17.4	7.1
MiniMax-M2.7	4.3	0.279	126.2	41	0.0	0.0	4.0	13.0	7.1
Seed-2.0-Pro	2.6	0.135	55.9	20	0.0	0.0	12.0	0.0	0.0

to credit partial completions: $CS = \frac{1}{N} \sum_t s_t$. We also report *Avg. turns*, the mean number of agent turns per task, and *Output Tok.*, the average output tokens generated per task (in thousands), as indicators of interaction cost and computational effort.

4.2 Main Results

Table 2 reports resolved rate, Completion Score, average turns, and per-domain resolved rates for thirteen frontier models under OpenHands, with Terminus 2 results for a subset of models alongside for comparison. A detailed scaffold sensitivity analysis is provided in §5.4.

Overall performance. Current frontier models remain far from solving ROADMAPBENCH. Under OpenHands, Claude-Opus-4.7 achieves the highest resolved rate at 39.1%, followed by Claude-Opus-4.6 at 32.2% and GPT-5.4 at 29.6%. The remaining ten models range from 5.2% to 20.9%, indicating a

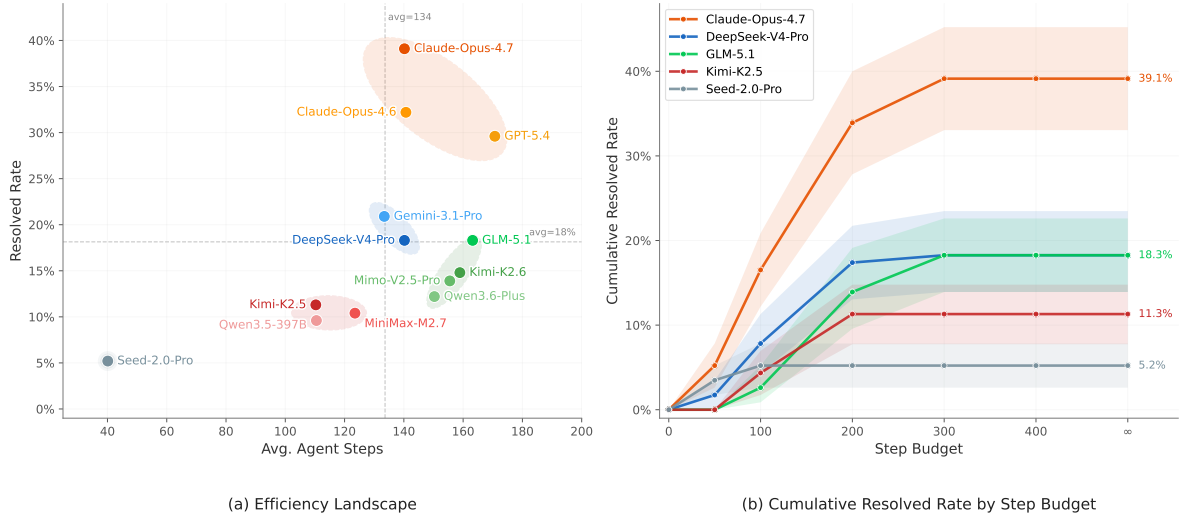


Figure 5: **Efficiency and step-budget analysis.** (a) Efficiency landscape of resolved rate versus average agent steps. Dashed lines mark fleet means, and shaded ellipses indicate performance tiers. (b) Cumulative resolved rate under increasing per-task step budgets, showing how models convert additional compute into task resolution.

substantial gap between the strongest models and the rest. Completion Score highlights that partial progress is common. It is consistently higher than resolved rate across models, showing that agents often complete some roadmap targets before failing to solve the full task. For example, Claude-Opus-4.6 resolves 32.2% of tasks but obtains a Completion Score of 0.627, while Seed-2.0-Pro resolves 5.2% yet reaches 0.177. This suggests that failures often occur after partial progress, when agents stall on later targets, integration, or correctness.

Domain difficulty. Performance varies substantially across domains. ML & Data is the most challenging: six of thirteen models resolve no tasks, and only the top three exceed 8%. ORM & Validation is relatively more tractable, likely due to the structured nature of schema migration and validation APIs. UI & Rendering shows the sharpest separation across capability tiers, with Claude-Opus-4.7 reaching 64.3% and Claude-Opus-4.6 reaching 57.1%, while weaker models remain much lower. Web & RPC and Infra. & Tooling fall between these extremes, reflecting intermediate levels of domain structure and integration complexity.

5 Analysis

We decompose performance along six aspects: **Step Efficiency and Compute Scaling** (§5.1), capturing how much trajectory budget is consumed per resolved task; **Tool Composition and Usage Distribution** (§5.2), capturing how that budget is allocated across different intents; **Task Complexity and Performance** (§5.3), examining how complexity affects resolution; **Scaffold Sensitivity** (§5.4), comparing agent frameworks; **Target-Level Analysis** (§5.5), stratifying by change type and difficulty; and **Failure Mode Analysis** (§5.6), characterizing where unsuccessful trajectories break down.

5.1 Step Efficiency and Compute Scaling

To characterize behavioral patterns and step efficiency across models, Figure 5(a) plots average agent steps against resolved rate, with dashed lines marking the fleet averages of 134 steps and 18%. The models separate into distinct regimes. Frontier models, including Claude-Opus-4.7, Claude-Opus-4.6, and GPT-5.4, achieve 30% to 39% resolved rates with moderate to high step budgets. By contrast, models such as GLM-5.1 and Kimi-K2.6 consume comparable or larger budgets but remain near the mid-performance

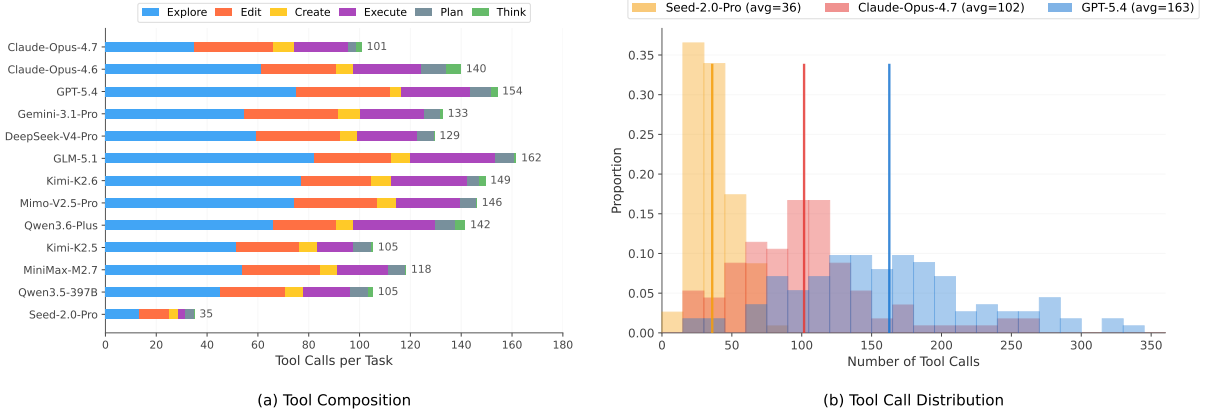


Figure 6: **Tool usage analysis.** (a) Tool composition by model, decomposed into six intent categories and sorted by resolved rate. (b) Distribution of per-task tool call counts for three representative models spanning the full performance range: Seed-2.0-Pro (5%), Claude-Opus-4.7 (39%), and GPT-5.4 (30%). Vertical lines indicate mean values.

region, indicating lower step efficiency. This contrast is particularly clear for Claude-Opus-4.7 and GLM-5.1, which use similar average budgets, 140 and 163 steps respectively, yet differ by more than 20 percentage points in resolved rate. Seed-2.0-Pro appears as a low-compute, low-performance outlier, suggesting premature termination or limited repository interaction.

Figure 5(b) shows the cumulative resolved rate as the per-task step budget increases. Most models saturate within the first 200 steps, after which additional budget provides limited gains. The strongest model, Claude-Opus-4.7, is the main exception, continuing to improve beyond this point and reaching 39.1% at the full budget. Among mid-tier models, DeepSeek-V4-Pro and GLM-5.1 reach similar final resolved rates but follow different scaling trajectories. DeepSeek-V4-Pro plateaus earlier, indicating higher step efficiency, whereas GLM-5.1 requires a larger budget to approach the same level. These trends suggest that additional steps are beneficial only when models can effectively convert longer trajectories into successful edits.

5.2 Tool Composition and Usage Distribution

We classify each tool invocation into six intent-based categories derived from the OpenHands agent’s action space. *Explore* encompasses file viewing (`str_replace_editor view`) and shell-based search or inspection commands (e.g., `grep`, `find`, `cat`); *Edit* covers in-place code modifications (`str_replace_editor str_replace`) and shell editing commands; *Create* captures new file creation (`str_replace_editor create/insert`); *Execute* includes compilation, testing, dependency installation, and other shell executions; *Plan* corresponds to explicit task planning via the built-in task tracker; and *Think* represents deliberate reasoning steps. Terminal actions (e.g., task completion) and tool misuse are excluded.

As shown in Figure 6(a), *Explore*, *Edit*, and *Execute* dominate tool usage across all models, corresponding to repository inspection, code modification, and validation. The main difference across models is not the amount of tool use, but how tool calls are allocated across the development process. Claude-Opus-4.7 achieves the highest resolved rate with only 101 tool calls per task on average and the lowest *Explore* ratio at 35%. In contrast, GLM-5.1 and Kimi-K2.6 use substantially more tool calls, but spend over half of them on exploration. This suggests that strong models localize relevant code more efficiently and shift earlier from exploration to targeted editing and execution-based validation. Explicit *Plan* and *Think* calls remain sparse for most models, indicating that the observed trajectories are driven mainly by iterative exploration, editing, and execution rather than dedicated reasoning-oriented tool actions.

Figure 6(b) compares the per-task tool call distributions of three representative models. Seed-2.0-Pro

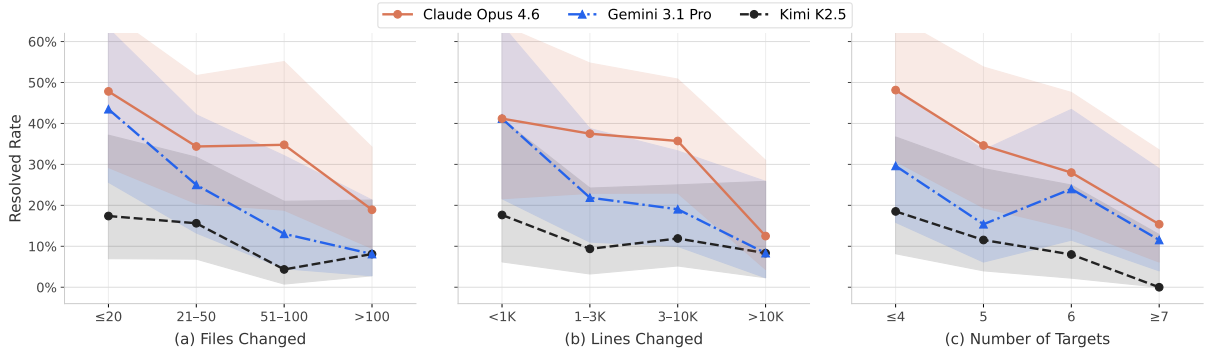


Figure 7: **Resolved rate vs. three task complexity proxies** (binned rate $\pm$ 95% Wilson CI). (a) Files changed, (b) lines changed, and (c) number of targets are all strong predictors of task difficulty, with monotonically decreasing resolved rates as complexity increases.

uses only 36 tool calls on average and obtains a low resolved rate, suggesting insufficient repository interaction. GPT-5.4 uses 163 tool calls on average, indicating much longer trajectories. Claude-Opus-4.7 reaches the best resolved rate with an intermediate average of 102 tool calls. Overall, these results indicate that task success is better characterized by the allocation of tool use across exploration, editing, planning, and execution than by raw tool-call volume alone.

5.3 Task Complexity and Performance

Resolved rate declines consistently as task complexity increases across all three structural proxies (Figure 7). *Files changed* (a) shows the clearest model separation: stronger models hold up longer as file count grows, while weaker models fall off early, with Gemini dropping from 43% to 8% across the full range—a steeper decline than Claude’s 48% to 19%. *Code volume* (b) reveals a more nuanced pattern: on simpler tasks (under 1K lines), Claude and Gemini start at similar levels ($\sim$ 41%), but Claude maintains a clear advantage through mid-range complexity while Gemini drops sharply in the intermediate bins; at the hardest end (>10 K lines), both converge near the floor, suggesting extreme complexity is a ceiling even for the strongest models. *Subtask count* (c) amplifies this dynamic most dramatically: Kimi-K2.5 collapses to 0% at 7 or more subtasks while Claude still resolves 15%, making it the sharpest discriminator among the three proxies. Together, these results confirm that structural complexity is an effective performance discriminator, with the sharpest separation occurring in the mid-range where model capabilities diverge most.

5.4 Scaffold Sensitivity

Performance varies across scaffolds for most models, but the direction and magnitude differ by capability tier. Three patterns emerge from Table 2.

Top models are scaffold-robust. Claude-Opus-4.6 achieves 31.3% on Terminus 2 and 32.2% on OpenHands, a difference of 0.9 percentage points. Mid- and lower-tier models show larger swings of 3 to 10 percentage points across scaffolds.

OpenHands yields higher performance for most models. The majority of evaluated models perform better under OpenHands. The gains are largest for DeepSeek-V4-Pro (+7.9 pp) and MiniMax-M2.7 (+6.1 pp). OpenHands provides explicitly typed tool schemas with clear argument names, which reduces the effort required to select and format each tool call correctly.

Two models perform better on Terminus 2. GLM-5.1 and Qwen3.6-Plus are the only exceptions, with resolved rates 2.6 pp and 4.3 pp higher on Terminus 2. Terminus 2 requires the agent to batch multiple

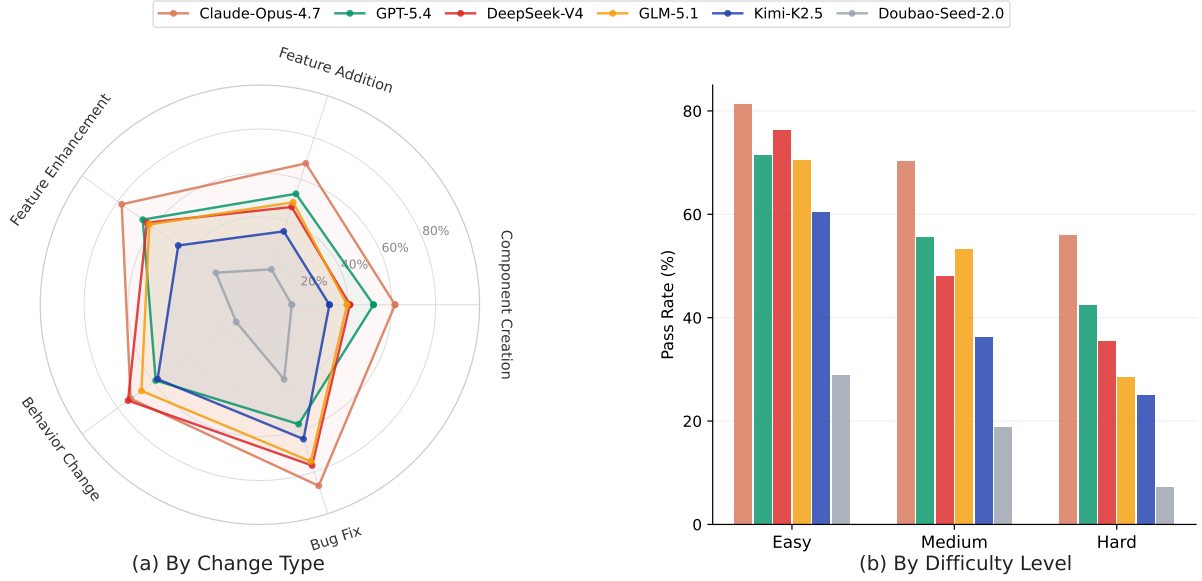


Figure 8: **Subtask pass rate for six representative models.** (a) By change type. (b) By difficulty level.

commands into a single structured JSON response per turn, a format these two models handle more effectively than the one-action-per-turn interface of OpenHands.

5.5 Target-Level Analysis

We classify subtasks into five change types: Component Creation, Feature Addition, Feature Enhancement, Behavior Change, and Bug Fix. A clear difficulty gradient emerges: average pass rate rises from 36% (Component Creation) to 64% (Bug Fix), confirming that designing new abstractions and multi-file coordination is substantially harder than locating and correcting specific defects.

Figure 8 breaks down performance by change type and difficulty level across six representative models. Panel (a) reveals that the gap between strong and weak models is most pronounced on Component Creation and Feature Addition, where Claude maintains over 50% while Doubao drops below 25%. Panel (b) shows that on Hard subtasks, Claude maintains 53% while DeepSeek drops to 43% and Doubao to 16%, confirming that difficulty amplifies inter-model gaps.

5.6 Failure Mode Analysis

We perform root-cause analysis on 3,603 failed subtasks across thirteen models using Claude-Sonnet-4.6 as an agentic classifier. We categorize failures into five types. Implementation Error refers to code that compiles but exhibits incorrect behavior. Build Error denotes solutions that fail to compile or link. Missing Implementation captures cases where required functionality is absent. Interface Mismatch covers incorrect API signatures or export paths. Agent Failure refers to cases where the agent abandons the task or exhausts its budget.

Figure 9 reveals a capability-dependent shift in failure modes. Higher-performing models are less often blocked by construction-level errors such as build failures or missing functionality; instead, their failures concentrate on implementation-level correctness. For Claude-Opus-4.6, 58% of failures are Implementation Errors, indicating that the model usually produces complete and buildable code but still fails on behavioral correctness. These errors are further dominated by Code Defect, Misunderstanding, and Wiring Error, suggesting that the frontier bottleneck lies in execution precision, including subtle logic mistakes, requirement misinterpretation, and component integration. Gemini-3.1-Pro presents a transitional profile, with Build Error and Implementation Error contributing comparable shares, 38%

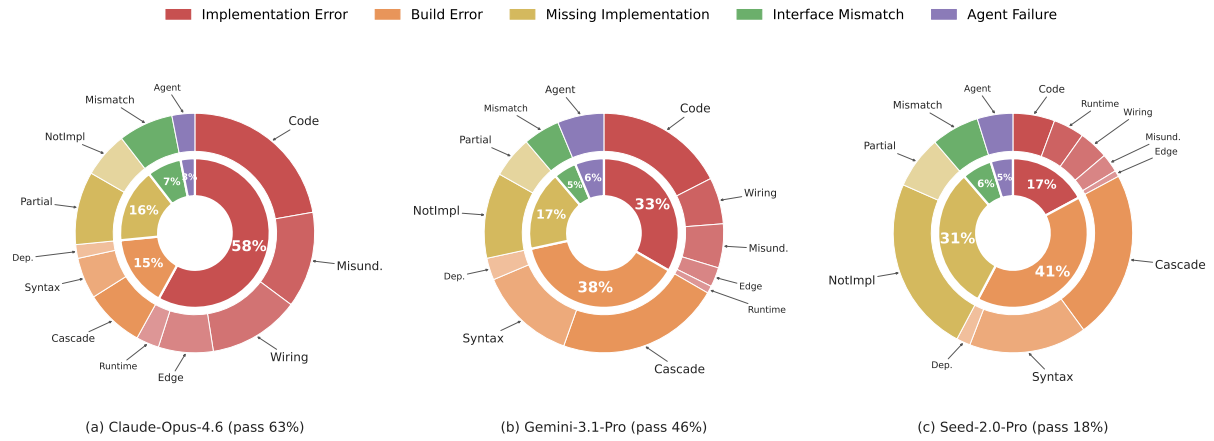


Figure 9: **Error distribution for three representative models.** Inner ring: category proportions; outer ring: sub-type breakdown. The dominant failure mode shifts from Implementation Error (strong models) to Build Error (weak models).

and 33%, respectively. Seed-2.0-Pro is dominated by earlier construction failures, with Build Error and Missing Implementation accounting for 41% and 31% of failures. This pattern indicates that, as model capability decreases, the primary bottleneck shifts from implementing the correct behavior to producing complete and buildable code.

6 Conclusion

ROADMAPBENCH introduces a new evaluation axis for coding agents: multi-target, long-horizon software development across real version upgrades. Each task requires agents to interpret roadmap specifications, coordinate multi-file changes, and implement coherent feature sets. Across 115 tasks from 17 repositories and 5 programming languages, current models remain far from solving this setting. Under OpenHands, Claude-Opus-4.7 resolves only 39.1% of tasks, while Seed-2.0-Pro resolves 5.2%. Completion Score shows that partial progress is common: agents often complete a subset of roadmap targets before failing on integration, correctness, or construction-level reliability. Domain-level results further show uneven difficulty, with ML & Data being the most challenging, ORM & Validation relatively more tractable, and UI & Rendering exhibiting a large gap between frontier and weaker models. Our analysis indicates that stronger models more efficiently localize relevant code and convert exploration into targeted edits, whereas weaker models often fail earlier through build errors or missing implementations. These results position ROADMAPBENCH as a diagnostic benchmark for measuring sustained software development capability beyond isolated issue resolution.

References

- OpenAI. SWE-bench Verified. Technical report, OpenAI, 2024.
- Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Laufer, Andrew Park, Nitin Pasari, Chetan Rane, et al. SWE-bench pro: Can ai agents solve long-horizon software engineering tasks? *arXiv preprint arXiv:2509.16941*, 2025.
- Qixing Zhou, Jiacheng Zhang, Haiyang Wang, et al. FeatureBench: Benchmarking agentic coding for complex feature development. *arXiv preprint arXiv:2602.10975*, 2025.
- Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason

-
- Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026.
- Minh VT Thai, Tue Le, Dung Nguyen Manh, Huy Phan Nhat, and Nghi DQ Bui. Swe-evo: Benchmarking coding agents in long-horizon software evolution scenarios. *arXiv preprint arXiv:2512.18470*, 2025.
- Jingzhe Ding, Shengda Long, Changxin Pu, et al. NL2Repo-Bench: Towards long-horizon repository generation evaluation of coding agents. *arXiv preprint arXiv:2512.12730*, 2025.
- Anthropic. Claude opus 4.6. *Anthropic Blog Post*, 2026a. <https://www.anthropic.com/claude>.
- OpenAI. Introducing gpt-5.4. *OpenAI Blog Post*, 2026. <https://openai.com/index/introducing-gpt-5-4/>.
- Google DeepMind. Gemini 3 flash model card. <https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-3-Flash-Model-Card.pdf>, 2025.
- John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.
- Kechi Zhang, Jia Li, Ge Li, Xianjie Shi, and Zhi Jin. Codeagent: Enhancing code generation with tool-integrated agent systems for real-world repo-level coding challenges, 2024. URL <https://arxiv.org/abs/2401.07339>.
- Siming Huang, Tianhao Cheng, Jason Klein Liu, Weidi Xu, Jiaran Hao, Liuyihan Song, Yang Xu, Jian Yang, Jiaheng Liu, Chenchen Zhang, et al. Opencoder: The open cookbook for top-tier code large language models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 33167–33193, 2025.
- Haoran Wang, Zhenyu Hou, Yao Wei, Jie Tang, and Yuxiao Dong. Swe-dev: Building software engineering agents with training and inference scaling. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 3742–3761, 2025.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.
- Tianyang Liu, Canwen Xu, and Julian McAuley. Repobench: Benchmarking repository-level code auto-completion systems. *arXiv preprint arXiv:2306.03091*, 2023.
- Xueying Du, Mingwei Liu, Kaixin Wang, Hanlin Wang, Junwei Liu, Yixuan Chen, Jiayi Feng, Chaofeng Sha, Xin Peng, and Yiling Lou. Classeval: A manually-crafted benchmark for evaluating llms on class-level code generation. *arXiv preprint arXiv:2308.01861*, 2023.
- Ranjan Sapkota, Konstantinos I Roumeliotis, and Manoj Karkee. Vibe coding vs. agentic coding: Fundamentals and practical implications of agentic ai. *arXiv preprint arXiv:2505.19443*, 2025.
- Yihong Dong, Xue Jiang, Jiaru Qian, Tian Wang, Kechi Zhang, Zhi Jin, and Ge Li. A Survey on Code Generation with LLM-based Agents, 2025. URL <https://arxiv.org/abs/2508.00083>.
- Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, Rachel Dias, Evan Mays, Benjamin Kinsella, Wyatt Thompson, et al. PaperBench: Evaluating AI’s Ability to Replicate AI Research. *arXiv preprint arXiv:2504.01848*, 2025.
- Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. Openhands: An open platform for ai software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.

Anthropic. Claude Code. <https://www.anthropic.com/claude-code>, 2025. Accessed: 2026-04-26.

Mark Chen, Jerry Tworek, Heewoo Jun, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Jacob Austin, Augustus Odena, Maxwell Nye, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Qixing Zhou, Jiacheng Zhang, Haiyang Wang, Rui Hao, Jiahe Wang, Minghao Han, Yuxue Yang, Shuzhe Wu, Feiyang Pan, Lue Fan, et al. Featurebench: Benchmarking agentic coding for complex feature development. *arXiv preprint arXiv:2602.10975*, 2026.

Anthropic. Claude opus 4.7. *Anthropic Blog Post*, 2026b. <https://www.anthropic.com/news/claude-opus-4-7>.

Google DeepMind. Gemini 3.1 Pro. <https://deepmind.google/models/gemini/pro/>, 2026. Accessed: 2026-04-29.

DeepSeek-AI. DeepSeek-V4: Towards highly efficient million-token context intelligence, 2026.

GLM-5-Team. GLM-5: From vibe coding to agentic engineering. *arXiv preprint arXiv:2602.15763*, 2026.

MiniMax. Kimi-K2.6. <https://www.kimi.com/blog/kimi-k2-6>, 2026a. Accessed: 2026-04-23.

XiaoMi. Xiaomi MiMo-V2.5-Pro. <https://mimo.xiaomi.com/mimo-v2-5-pro>, 2026. Accessed: 2026-04-27.

Qwen Team. Qwen3.6-Plus: Towards real world agents. <https://qwen.ai/blog?id=qwen3.6>, 2026a. Accessed: 2026-04-29.

Kimi Team. Kimi K2.5: Visual agentic intelligence. *arXiv preprint arXiv:2602.02276*, 2026.

MiniMax. MiniMax-M2.7. <https://www.minimax.io/models/text/m27>, 2026b. Accessed: 2026-04-29.

Qwen Team. Qwen3.5-397B. <https://artificialanalysis.ai/articles/qwen3-5-397b-a17b-everything-you-need-to-know>, 2026b. Accessed: 2026-02-17.

ByteDance Seed Team. Seed-2.0. <https://seed.bytedance.com/en/seed2>, 2026. Accessed: 2026-04-29.

Harbor Framework Team. Harbor: A framework for evaluating and optimizing agents and models in container environments, jan 2026. URL <https://github.com/harbor-framework/harbor>.

Appendix

A Limitations

We acknowledge several limitations of this work. Our evaluation employs two agent scaffolds (OpenHands and Terminus 2). Agent performance is sensitive to scaffold design choices, and results under other frameworks may differ. Evaluation relies on test suites that verify behavioral correctness but do not assess code quality, maintainability, or adherence to idiomatic patterns. Future work could incorporate multi-dimensional metrics for a more holistic assessment. Although ROADMAPBENCH spans five programming languages and multiple software domains, it still covers only a limited subset of real-world development ecosystems. Future extensions could incorporate additional languages, frameworks, and application settings.

B Ethics Statement

This research conforms to the Code of Ethics. All benchmark tasks are derived from publicly available open-source repositories. No private or proprietary code is included. Repository identities and version numbers are anonymized in the task instructions to prevent information leakage during evaluation, and no personally identifiable information is collected or used. Human annotators involved in quality control are co-authors of this work and participated voluntarily.

C Broader Impacts

ROADMAPBENCH is designed to measure and advance the capability of coding agents on realistic software engineering tasks. On the positive side, improved coding agents can increase developer productivity, lower barriers to software development, and accelerate open-source contributions. On the negative side, more capable coding agents could potentially be misused to generate malicious code or exploit vulnerabilities at scale. However, our benchmark evaluates agents on constructive software development tasks (implementing features from public roadmaps) rather than adversarial capabilities. We do not release any model weights or fine-tuning recipes. We believe the diagnostic value of understanding where current agents fail outweighs the marginal risk, as the benchmark primarily reveals limitations rather than enabling new harmful capabilities.

D Human Evaluation

We conduct human evaluation as part of the task construction and quality-control pipeline. All annotators are Ph.D. students with computer science backgrounds and relevant experience in software engineering. They participated in constructing coding tasks, reviewing generated instructions, and repairing task-side defects identified during validation. The annotators were compensated above the local minimum hourly wage.

E Task Details

This appendix provides detailed statistics that supplement the dataset overview in Section 3.2.

E.1 Repository and Language Coverage

Table 3 summarizes the repository coverage of our benchmark across five programming languages and diverse software domains.

Table 3: Repository coverage by language, domain, task count, and median oracle-patch complexity.

Language	Repository	Tasks	Domain	Med. Lines	Med. Files	Med. Subtasks
Python	Polars	13	ML & Data	1,346	42	5
	PyG	10	ML & Data	7,044	140	6
	Optuna	8	ML & Data	4,054	82	5
	spaCy	5	ML & Data	4,226	135	6
	Falcon	5	Web & RPC	3,311	44	6
TypeScript	MikroORM	10	ORM & Val	6,006	122	6
	Prisma	9	ORM & Val	1,246	30	4
	Valibot	3	ORM & Val	3,341	56	5
C++	Glaze	14	Infra & Tool	3,745	29	5
	thread-pool	6	Infra & Tool	1,065	2	4
Go	Fiber	6	Web & RPC	1,997	25	6
	Kitex	6	Web & RPC	8,018	149	5
	Fyne	5	UI & Ren	20,339	876	7
Rust	Ratatui	6	UI & Ren	6,575	44	6
	Diesel	3	ORM & Val	9,233	169	4
	Slint	3	UI & Ren	1,656	29	4
	Ruff	3	Infra & Tool	17,130	357	6
Overall (17 repos)		115		3,714	51	5

E.2 Task Complexity Distribution

Figure 10 plots all 115 tasks in lines-changed vs. files-changed space. The leftmost panel shows the full benchmark with dashed reference lines at the medians (3,714 lines, 51 files). The remaining five panels facet the data by programming language, highlighting each language against the full benchmark. The strong positive correlation confirms that tasks requiring more code also touch more files, and the spread over two orders of magnitude in both dimensions demonstrates the benchmark’s diversity. Python tasks cluster in a moderate range with several high-complexity outliers, while Go and Rust tasks tend toward high file counts due to generated code and macro expansions.

Figure 11 presents vertical boxplots of files changed per repository, sorted by median. The log-scale y-axis highlights that complexity varies by more than two orders of magnitude across the benchmark.

E.3 Temporal Span and Repository Scale

Figure 12 visualizes the version upgrade trajectories as a constellation plot. Each line segment connects a task’s source version release to its target version, positioned by release date (x-axis) and repository source size (y-axis, log scale). Solid lines indicate that the codebase grew between versions; dashed lines indicate code cleanup (net size reduction).

The benchmark spans releases from 2017 to 2026, covering nearly a decade of software evolution. Repository sizes range from ~ 20 KB (thread-pool) to ~ 10 MB (Polars, Go repositories), demonstrating diversity across small libraries and large codebases. The temporal spread reduces the risk of memorization from training data.

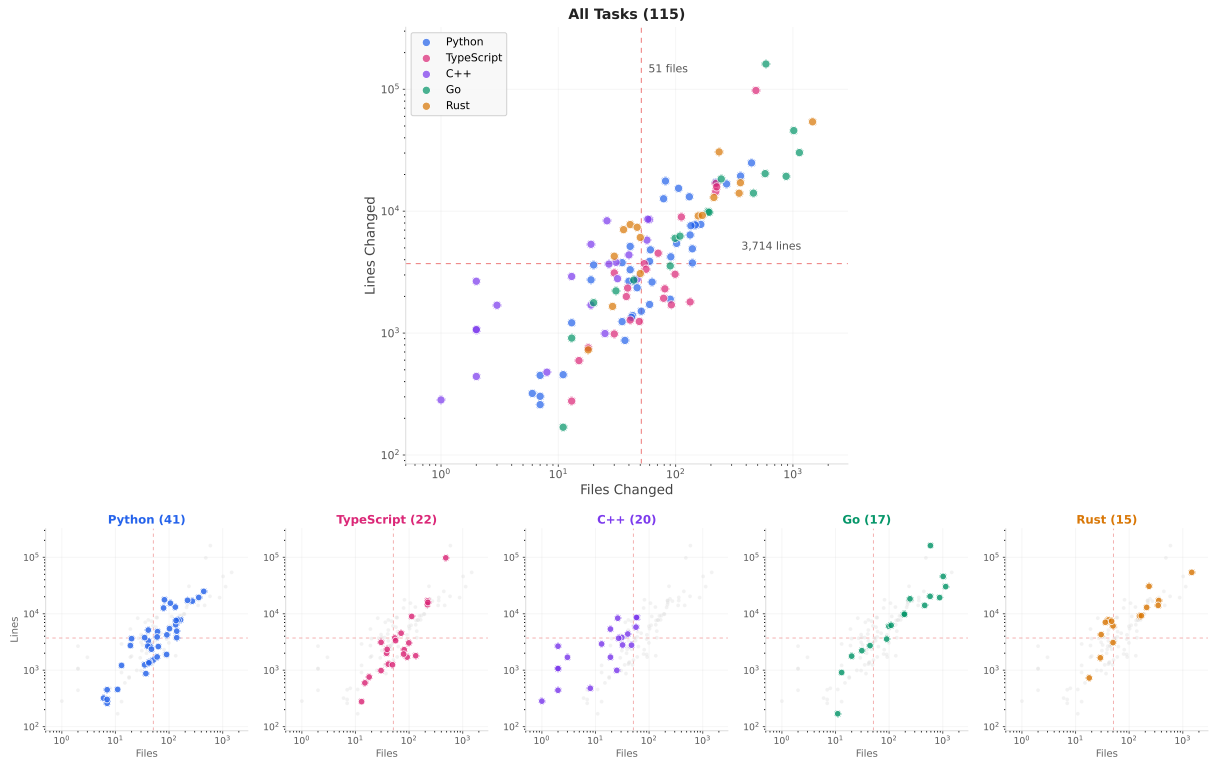


Figure 10: **Task complexity overview and per-language breakdown** (log-log scale). (a) All 115 tasks colored by language, with dashed lines at the benchmark medians (3,714 lines, 51 files). (b) Per-language panels: each language highlighted against the full benchmark (gray).

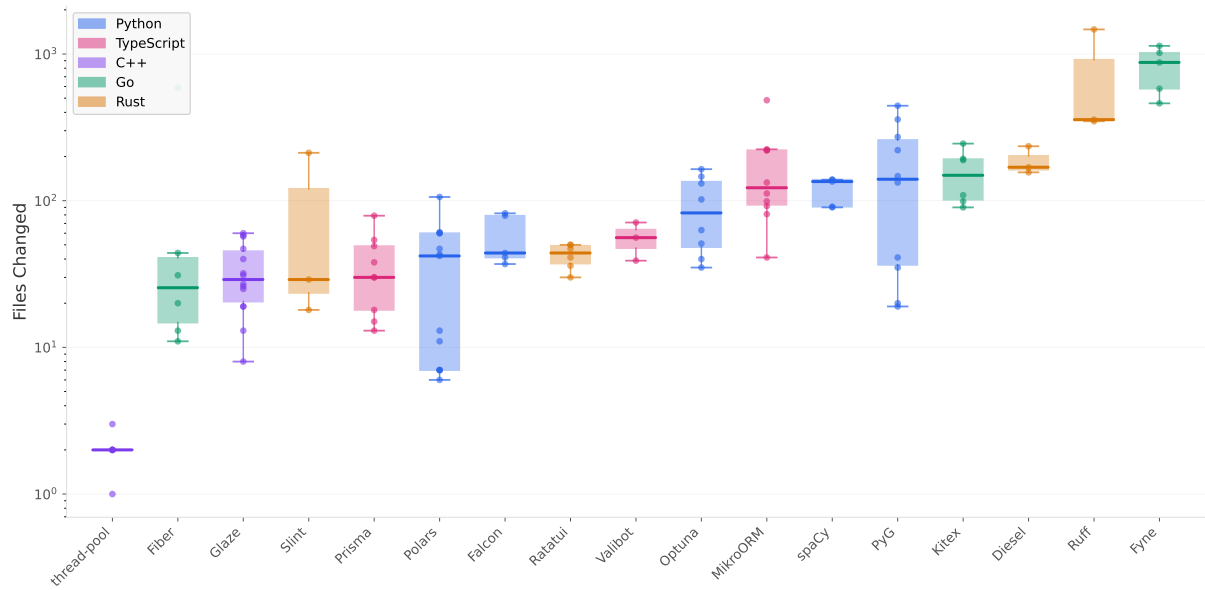


Figure 11: **Distribution of files changed (oracle patch) across repositories**. Repos are sorted by median files changed (log scale); individual task values are shown as jittered points.

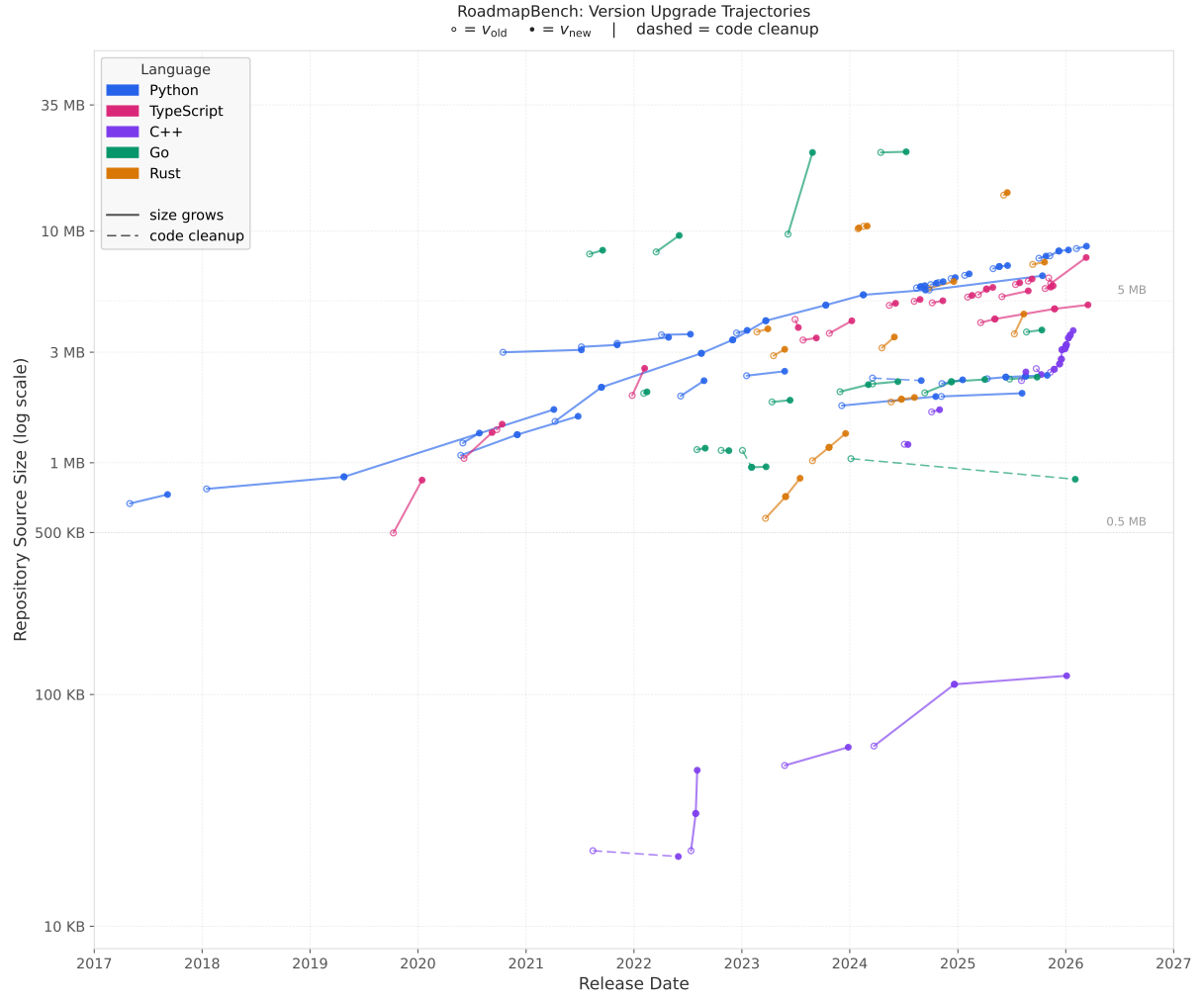


Figure 12: **Version upgrade trajectories.** Each segment represents one task: hollow circles mark the source version, filled dots mark the target version. Solid lines indicate codebase growth; dashed lines indicate net size reduction. The temporal spread (2017–2026) and size diversity (20 KB–10 MB) demonstrate broad benchmark coverage.

F Task Example

Below is the complete instruction for `opt-4.0.0-roadmap`, a representative ROADMAPBENCH task grounded in the real Optuna v3.6.0→v4.0.0 transition (164-day window, oracle patch: 164 files, 7,794 LOC filtered).

Hyperparameter Optimization Framework Development Roadmap

Overview. This library is a hyperparameter optimization framework widely used in machine learning and scientific computing. It provides automated search over parameter spaces using efficient sampling algorithms, distributed optimization via various storage backends, and rich visualization of optimization results.

Goals. Stabilize two experimental subsystems: the artifact management system and the journal-based distributed storage backend. Introduce constrained optimization awareness into `best_trial` and `best_trials`. Add two new termination components — `EMMEvaluator` and `MedianErrorEvaluator`. Add `is_exhausted()` to the grid search sampler.

Target 1: Artifact Store Official APIs

Objective functions often produce files (model snapshots, logs) that need tracking alongside trial metadata. This target stabilizes the upload API and introduces `download_artifact` and `get_all_artifact_meta`.

Requirements

1. `ArtifactMeta` — frozen dataclass importable from `optuna.artifacts` with fields: `artifact_id: str`, `filename: str`, `mimetype: str`, `encoding: str | None`.
2. `download_artifact` — importable from `optuna.artifacts`. All parameters keyword-only: `artifact_store` (`ArtifactStore`), `file_path` (`str`), `artifact_id` (`str`). Returns `None`. Raises `FileExistsError` if `file_path` already exists.
3. `get_all_artifact_meta` — importable from `optuna.artifacts`. Positional: `study_or_trial` (`Trial`, `FrozenTrial`, or `Study`). Keyword-only: `storage` (default `None`, required for `FrozenTrial`). Returns `list[ArtifactMeta]`. Raises `ValueError` if `storage` is `None` and input is a `FrozenTrial`. When given a `Study`, returns only study-level artifacts.
4. `upload_artifact` — updated to use keyword-only parameters in order: `artifact_store`, `file_path`, `study_or_trial`, with `storage`, `mimetype`, `encoding` as additional keyword-only. Backward compatibility with old positional order maintained. Returns `str` (artifact ID). Infers MIME type from file extension; defaults to `"application/octet-stream"`.
5. `optuna.artifacts.__all__` must include: `ArtifactMeta`, `FileSystemArtifactStore`, `Boto3ArtifactStore`, `GCSArtifactStore`, `Backoff`, `get_all_artifact_meta`, `upload_artifact`, `download_artifact`.

Target 2: JournalStorage API Reorganization

The journal-based storage backend enables distributed optimization over NFS by recording operation logs instead of state snapshots. The module is being reorganized from a private location to a public subpackage with clearer naming conventions.

After this target, users should import journal components from `optuna.storages.journal` using the new class names, while old names remain available (with deprecation warnings) from `optuna.storages`.

Requirements

1. Create public subpackage `optuna/storages/journal/` containing: `__init__.py`, `_base.py`, `_file.py`, `_redis.py`, `_storage.py`.
2. Class renames (importable from `optuna.storages.journal`): `BaseJournalBackend` (was `BaseJournalLogStorage`), `BaseJournalSnapshot` (was `BaseJournalLogSnapshot`), `JournalFileBackend` (was `JournalFileStorage`), `JournalRedisBackend` (was `JournalRedisStorage`), `JournalFileSymlinkLock`, `JournalFileOpenLock`, `JournalStorage` (unchanged).
3. Old names remain importable from `optuna.storages` with deprecation warnings. `BaseJournalLogStorage` should subclass `BaseJournalBackend` decorated with `@deprecated_class`.
4. `optuna.storages.journal.__all__` must include: `JournalFileBackend`, `BaseJournalBackend`, `JournalFileOpenLock`, `JournalFileSymlinkLock`, `JournalRedisBackend`, `JournalStorage`.

Target 3: Constrained Optimization in Study Properties

When running constrained optimization, users set constraint values on each trial via system attributes. However, `best_trial` and `best_trials` currently ignore these constraints. This target makes these properties constraint-aware.

After this target, `study.best_trial` returns the best feasible trial (all constraint values ≤ 0.0), and `study.best_trials` computes the Pareto front from only feasible trials. If no feasible trials exist, `best_trial` raises `ValueError`.

Requirements

1. Helper module `optuna/study/_constrained_optimization.py`: define constant `_CONSTRAINTS_KEY = "constraints"`. Implement `_get_feasible_trials(trials)` returning only trials where all constraint values are ≤ 0.0 . Trials without a "constraints" key are considered infeasible.
2. `best_trial` property: if the best trial is infeasible, filter to feasible trials and select the one with best objective value (respecting `study.direction`). Raise `ValueError` if none exist.
3. `best_trials` property: when any trial has the constraints key, compute Pareto front from feasible trials only.

Target 4: New Terminator Algorithms

The existing termination framework allows optimization to stop when further trials are unlikely to yield improvements. This target introduces two new components: `EMMEvaluator` (Expected Minimum Model Regret) and `MedianErrorEvaluator` (derives threshold from paired improvement evaluator's outputs).

After this target, a user can create an `EMMEvaluator`, pair it with a `MedianErrorEvaluator`, and pass both to a Terminator for GP-based automatic stopping.

Requirements

1. `EMMEvaluator` — importable from `optuna.terminator`, inherits `BaseImprovementEvaluator`. Constructor: `__init__(self, deterministic_objective=False, delta=0.1, min_n_trials=2, seed=None)`. Raises `ValueError` if `min_n_trials <= 1`. Method `evaluate`: returns EMMR value; returns `sys.float_info.max` with insufficient trials or empty search space.
2. `MedianErrorEvaluator` — importable from `optuna.terminator`, inherits `BaseErrorEvaluator`. Constructor: `__init__(self, paired_improvement_evaluator, warm_up_trials=10, n_initial_trials=20, threshold_ratio=0.01)`. Raises `ValueError` for invalid args. Method `evaluate`: before sufficient data returns negative sentinel; on first sufficient call computes median of improvement values multiplied by `threshold_ratio`, caches result.
3. `optuna.terminator.__all__` must include both `EMMEvaluator` and `MedianErrorEvaluator`.

Target 5: GridSampler Exhaustion Check

When using `GridSampler`, the user may want to programmatically check whether all parameter combinations have been evaluated. Currently there is no public API for this.

Requirement

1. `is_exhausted(self, study: Study) -> bool` on `GridSampler`: returns `True` if all grid combinations have been evaluated, `False` otherwise.

Completion Criteria

- All new classes and functions importable from their documented paths
- Existing APIs remain unchanged (backward compatibility)
- Deprecated old names still importable with deprecation warnings
- Constraint-aware `best_trial` raises `ValueError` when no feasible trials exist
- `EMMEvaluator` returns finite values with sufficient trials and large values with insufficient data
- `MedianErrorEvaluator` caches its threshold after first computation
- `GridSampler.is_exhausted()` correctly reports grid coverage

G Construction Pipeline Details

G.1 Repository Selection Criteria

Candidate repositories must satisfy the following hard constraints: at least 1,000 GitHub stars, five or more tagged releases, continued release activity through 2025, and a primary language among our five targets (Python, TypeScript, Go, Rust, Java).

Definition of high-quality release documentation. We require that each selected repository maintains release documentation with sufficient information density to support task construction. Concretely, a release qualifies as high-quality if it satisfies the following criteria:

- Uses natural language to describe what changed in the version, rather than merely listing pull-request numbers or commit hashes.
- Explains the background or motivation behind non-trivial changes (e.g., “to address X limitation” or “in response to user feedback on Y”).
- Clearly states user-facing impacts such as breaking changes, deprecated APIs, behavioral modifications, or newly introduced features.
- Optionally includes code examples, configuration snippets, or migration guides (these are positive signals but not strictly required).

Releases that consist solely of auto-generated commit lists (e.g., `fix #123, merge PR #456`), empty bodies, or single-line descriptions are excluded. Each repository must have at least three releases meeting the above standard. Figure 13 shows representative examples of qualifying release documentation.

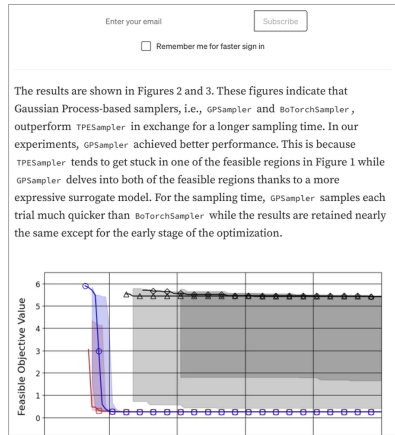
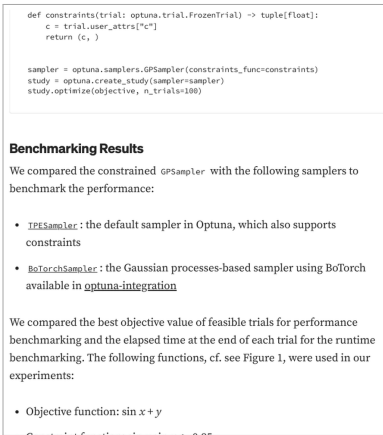
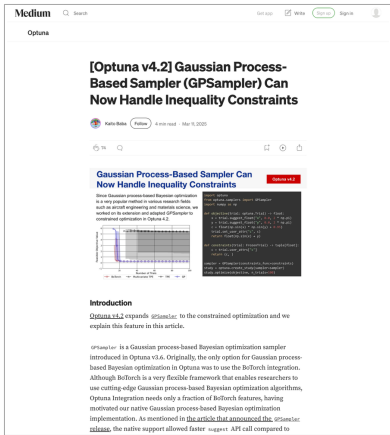
Expert review and version-pair selection. Expert reviewers verify the quality of release documentation identified in the previous step and select consecutive version pairs suitable for task construction. A version pair is retained if it satisfies: (1) a non-trivial code delta of at least 500 lines changed, (2) at least one externally visible behavioral change expressible as a deterministic test, and (3) release documentation that describes the change in sufficient detail to construct an instruction.

G.2 Static Review Details

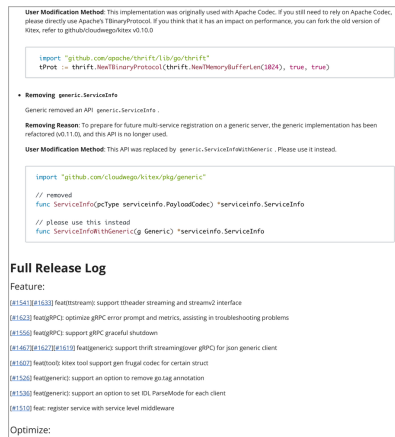
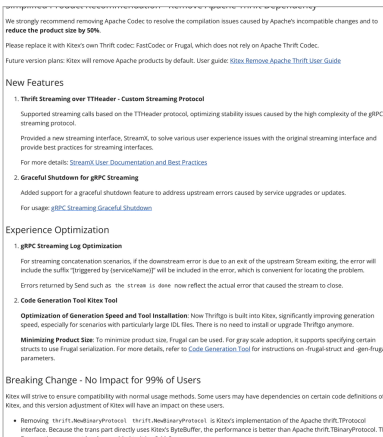
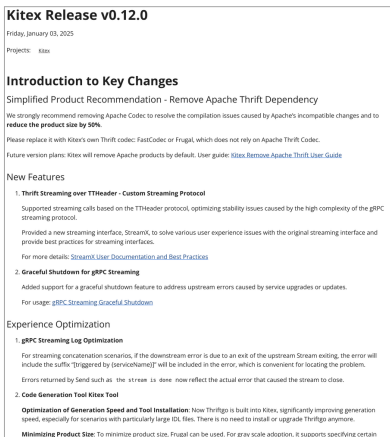
Stage 3 applies two complementary reviews under structured checklists, with expert reviewers assisted by Claude-Opus-4.7.

Compliance review (20 items). The compliance review is conducted from the perspective of a solver who has *no prior knowledge* of the repository or version upgrade. It covers five categories:

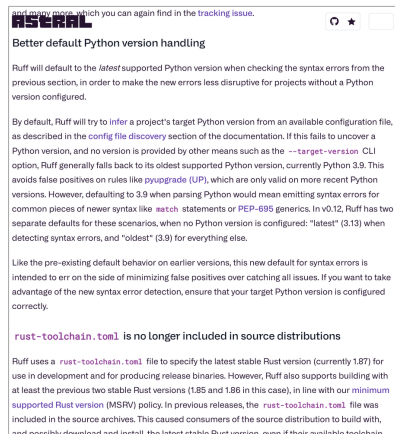
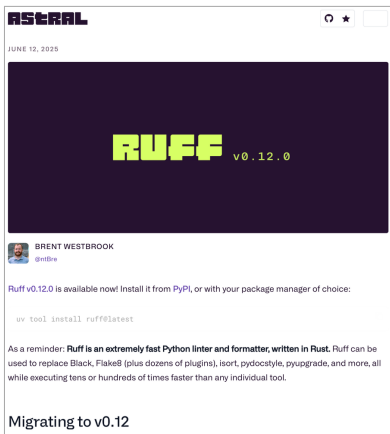
1. **Specification clarity** (Q1–Q3): each target’s goal and constraints are explicitly stated; the instruction is self-contained without referencing the construction process; requirements are defined positively rather than by exclusion.
2. **Implementation leakage** (Q4): a systematic scan for five leakage types—algorithm/flow steps, internal naming, pseudo-code control flow, bug root-cause disclosure, and refactoring checklists—that reveal *how* to implement rather than *what* behavior is required.
3. **Information integrity** (Q5–Q7): public API contracts are unambiguous; no test metadata (file names, function names, scoring details) is disclosed; no version numbers or repository names appear.
4. **Narrative quality** (Q8–Q9): the instruction provides a coherent version narrative with clear priority ordering among targets; individual target sections follow a consistent structure (background, requirements, constraints).
5. **Test conventions** (T1–T7): tests use the required directory layout; target weights sum to 1.0; tests are deterministic and environment-independent; tests do not check implementation internals beyond the specified public contract.



(a) Optuna v4.2 (medium.com/optuna)



(b) KiteX v0.12.0 (cloudwego.io)



(c) Ruff v0.12.0 (astral.sh)

Figure 13: Examples of high-quality release documentation from three selected repositories. Each row shows cropped excerpts from a single version release, illustrating feature narratives, code examples, migration guides, and breaking-change descriptions that serve as source material for task construction.

A task fails the compliance review if any item is marked FAIL. The synthesis agent revises the instruction or tests accordingly and re-validates.

Per-target correctness review. For each target independently, a reviewer checks instruction–test alignment along four dimensions:

1. **Completeness:** every behavior asserted by tests is stated in the instruction.
2. **Faithfulness:** tests do not assert behaviors beyond the instruction specification.
3. **Fairness:** tests do not rely on unstated assumptions (e.g., exact error wording, internal names).
4. **Minimality:** tests performing only dead-letter matching without behavioral value are flagged for removal.

Issues are classified as *T-missing*, *T-ambiguous*, *T-incorrect*, or *T-other*. Each confirmed issue is repaired by updating the instruction or test, and the oracle patch is re-run to confirm the fail-to-pass guarantee.

G.3 Quality Control Protocol

G.3.1 Attribution Classification

During rollout-based quality control, agent failures are attributed to either task-side defects (T-type) or model-side failures (M-type). T-type defects indicate problems in the task itself, such as missing specifications or flawed tests, while M-type failures reflect genuine limitations of the agent. Attribution is performed through expert review of agent trajectories and test outcomes.

T-type defects are classified into four subcategories: (1) instruction gaps (a behavioral requirement is not mentioned in the instruction), (2) test brittleness (a test assertion is stricter than the instruction warrants, e.g., checking internal implementation details), (3) environment issues (a dependency or environment variable required for the task is missing from the Docker image), and (4) grading errors (the subtask-level test runner assigns incorrect weights or groupings).

M-type failures are classified into three subcategories: (1) design failures (the agent’s implementation does not match the specification at a structural level), (2) implementation bugs (the implementation is structurally correct but contains code errors), and (3) debugging failures (the agent identifies an error but fails to correct it within the turn budget).

G.3.2 Inter-Annotator Agreement

To assess attribution consistency, we randomly sampled 40 agent trajectories for independent annotation by two annotators. Cohen’s κ for T-type vs. M-type classification was 0.83, indicating strong agreement. Disagreements were resolved by a third annotator. The classification rubric and calibration examples are included in the supplementary materials.

G.3.3 Iterative QC Impact

Each task undergoes iterative validation: an initial rollout identifies T-type defects, which are then fixed before re-evaluation. Of the 115 tasks, 45 required at least one fix round (average 3.1 rounds). Table 4 reports model performance before and after QC on all 115 tasks. For tasks that required no fix, the before and after scores are identical.

Table 4: Impact of iterative QC on model performance (Terminus, 115 tasks). “Before”: initial validation; “After”: post-repair rollout.

Model	Completion Score			Resolved (%)		
	Before	After	Δ	Before	After	Δ
Claude-Opus-4.6	0.564	0.683	+0.118	19.1	30.4	+11.2
GLM-5.1	0.475	0.511	+0.036	17.4	20.4	+3.0
Kimi-K2.5	0.329	0.348	+0.019	6.1	7.1	+1.0

H Error Classification Details

This appendix provides the complete error taxonomy, classification methodology, per-model distributions, and representative case studies referenced in §5.6.

H.1 Classification Methodology

Each failed subtask is classified by a Claude-Sonnet-4.6 instance operating in agentic mode via Claude Code. For each task containing failed subtasks, the classifier:

1. Reads the complete test output (`test-stdout.txt`) containing all subtask results.
2. Reads the task specification (`instruction.md`) to understand requirements.
3. Optionally inspects the agent’s final code or greps the trajectory for relevant context.
4. Outputs a structured classification for each failed subtask: category, sub-type, root-cause phrase (English, 2–5 words), and rationale (1–3 sentences with technical detail).

The task-level approach (one classifier call per task, classifying all failed subtasks together) enables cross-subtask awareness—e.g., recognizing that multiple subtasks fail due to the same root compilation error (classified as one primary *Syntax Error* plus cascading failures).

Validation. We manually validated 50 randomly sampled classifications across all models and categories. The automated classifier achieved 88% exact-match agreement with expert labels at the category level (following the validation protocol of [Jimenez et al. \(2024\)](#)). Disagreements primarily involved the boundary between *Code Defect* and *Wiring Error*—both are implementation-level failures, so category-level accuracy is higher than sub-type accuracy.

Coverage and cost. Classification covers 3,603 failed subtasks across 13 models (1,065 task groups). Total cost is approximately \$350.

H.2 Error Taxonomy

Table 5 defines the five error categories and fourteen sub-types. Categories are ordered by *failure stage*—from early catastrophic failures (code does not compile) to late subtle failures (code compiles and runs but produces incorrect results). Within each category, sub-types capture the specific mechanism of failure.

H.3 Per-Model Error Distribution

Figure 14 shows the aggregate error distribution across all 3,603 classified failures. Implementation Error is the dominant category (39%), followed by Build Error (28%) and Missing Implementation (23%). Within Implementation Error, Code Defect alone accounts for over half of the sub-type (23% overall).

Figure 15 breaks this down per model, ordered by subtask pass rate. Table 6 provides the exact counts and percentages.

H.4 Per-Model Analysis

Claude-Opus-4.7 (pass 70%). The strongest model with fewest total failures (136). Concentrates 55% in Implementation Error (Code Defect 38%), with Build Error nearly absent (4%). Missing Implementation accounts for 29%, driven equally by Not Implemented and Partially Implemented.

Claude-Opus-4.6 (pass 64%). Concentrates 58% of failures in Implementation Error, dominated by Code Defect (38%). Build Error is rare (15%), and nearly half of those are cascading failures from a single root cause. This model rarely leaves features unimplemented; its bottleneck is execution precision.

Table 5: **Error taxonomy with category and sub-type definitions.** Categories are ordered by failure stage. “Freq.” shows the distribution across all 3,603 classified failures.

Category	Sub-type	Definition	Freq.
Build Error (28.3%)	Cascading	A root error in a shared module causes compilation failure across multiple subtasks.	15.8%
	Syntax Error	Direct compilation/linking failure: syntax error, type mismatch, or unresolved symbol.	11.1%
	Dependency	Incompatible dependency version or import of an unavailable package.	1.4%
Missing Impl. (22.6%)	Not Implemented	Required functionality entirely absent—symbol or module does not exist.	13.8%
	Partially Impl.	Main feature exists but specific sub-requirements are skipped.	8.8%
Interface Mismatch (6.5%)	Wrong Signature	API exists but signature (parameters, return type) does not match.	3.7%
	Wrong Path	Code exists but is inaccessible: wrong module path or missing re-export.	2.7%
Impl. Error (38.5%)	Code Defect	Logical bug: wrong formula, off-by-one, nil dereference, incorrect condition.	23.1%
	Wiring Error	Components correct but integration broken: params not forwarded, features not activated.	6.8%
	Misunderstanding	Agent misinterprets the specification; implements wrong semantics.	5.9%
	Edge Case	Main path works; failures only on unusual boundary inputs.	1.4%
	Runtime Crash	Compiles but crashes at runtime: unhandled exception, deadlock, OOM.	1.4%
Agent Failure (4.0%)	Abandoned	Agent stops working: gives up, analysis paralysis, or skips remaining subtasks.	3.5%
	Exhausted	Budget/step/time limit hit or OOM-killed before completion.	0.5%

Table 6: **Per-model error category distribution** (count and percentage of failed subtasks). Parentheses after model names indicate subtask pass rate.

Model	Impl.	Build	Miss.	Intf.	Agent	Total
Claude-Opus-4.7 (70%)	75 (55%)	5 (4%)	40 (29%)	3 (2%)	13 (10%)	136
Claude-Opus-4.6 (64%)	94 (58%)	25 (15%)	26 (16%)	12 (7%)	5 (3%)	162
GPT-5.4 (55%)	55 (26%)	55 (26%)	76 (36%)	8 (4%)	20 (9%)	214
DeepSeek-V4-Pro (51%)	123 (51%)	54 (23%)	39 (16%)	17 (7%)	6 (3%)	239
GLM-5.1 (51%)	106 (46%)	66 (29%)	38 (17%)	12 (5%)	7 (3%)	229
Kimi-K2.6 (46%)	102 (36%)	89 (32%)	63 (22%)	12 (4%)	16 (6%)	282
Gemini-3.1-Pro (45%)	101 (33%)	116 (38%)	52 (17%)	15 (5%)	19 (6%)	303
Qwen3.6-Plus (42%)	131 (40%)	91 (28%)	71 (22%)	32 (10%)	4 (1%)	329
Kimi-K2.5 (38%)	132 (42%)	80 (26%)	69 (22%)	22 (7%)	9 (3%)	312
MiniMax-M2.7 (36%)	141 (44%)	79 (25%)	68 (21%)	32 (10%)	2 (1%)	322
Mimo-V2.5-Pro (36%)	135 (48%)	71 (25%)	48 (17%)	25 (9%)	5 (2%)	284
Qwen3.5-397B (35%)	111 (35%)	96 (31%)	78 (25%)	12 (4%)	16 (5%)	313
Seed-2.0-Pro (17%)	82 (17%)	194 (41%)	148 (31%)	31 (6%)	23 (5%)	478
Overall	1,388 (39%)	1,021 (28%)	816 (23%)	233 (6%)	145 (4%)	3,603

GPT-5.4 (pass 55%). Uniquely dominated by Missing Implementation (36%)—the highest among all models. Agent Failure is also elevated (9%, all Abandoned), reflecting the “analysis paralysis” pattern where the model explores extensively but never starts writing code. When it does implement, Build and Implementation Errors are balanced (26% each).

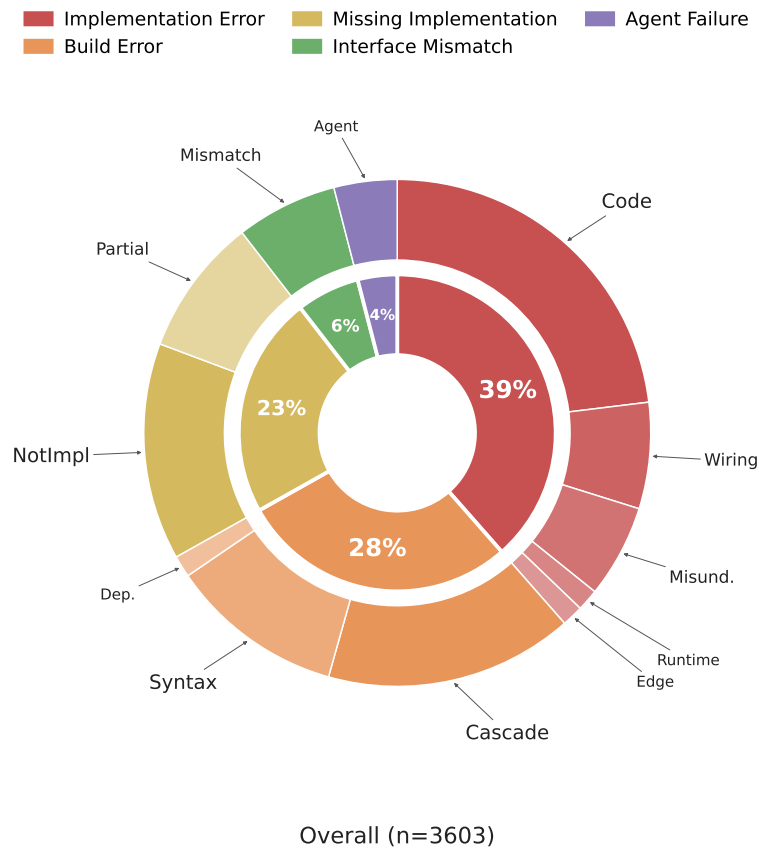


Figure 14: **Overall error distribution across all models** (n=3,603 failed subtasks). Implementation Error dominates (39%), with Code Defect as the single largest sub-type.

DeepSeek-V4-Pro (pass 51%). Profile resembles Opus but with more Build Errors (23% vs. 15%). Implementation Error remains dominant (51%), indicating strong architectural planning but less precise execution. Agent Failure is minimal (3%).

GLM-5.1 (pass 51%). Similar to DeepSeek with 46% Implementation Error and 29% Build Error. The higher Build Error ratio compared to Opus suggests less robust handling of complex type systems and module structures.

Kimi-K2.6 (pass 46%). Balanced between Implementation Error (36%) and Build Error (32%), with cascading failures accounting for 21% of total. Agent Failure is moderately elevated (6%), split between Abandoned (12) and Exhausted (4). Profile sits between GLM-5.1 and Gemini—stronger than its predecessor K2.5 on Implementation Error but with similar Build Error rates.

Gemini-3.1-Pro (pass 45%). Build Error dominates (38%)—the highest share among mid-tier models. Cascading failures are frequent (22% of total failures), indicating that compilation errors in early subtasks propagate to later subtasks. Implementation Error is relatively lower (33%).

Qwen3.6-Plus (pass 42%). Interface Mismatch is notably high (10%), suggesting difficulty with API surface compliance (export paths, naming conventions). Otherwise balanced between Implementation Error (40%) and Build Error (28%).

Kimi-K2.5 (pass 38%). Distribution closely matches Qwen3.6-Plus. Missing Implementation (22%) indicates that this model occasionally abandons complex sub-requirements.

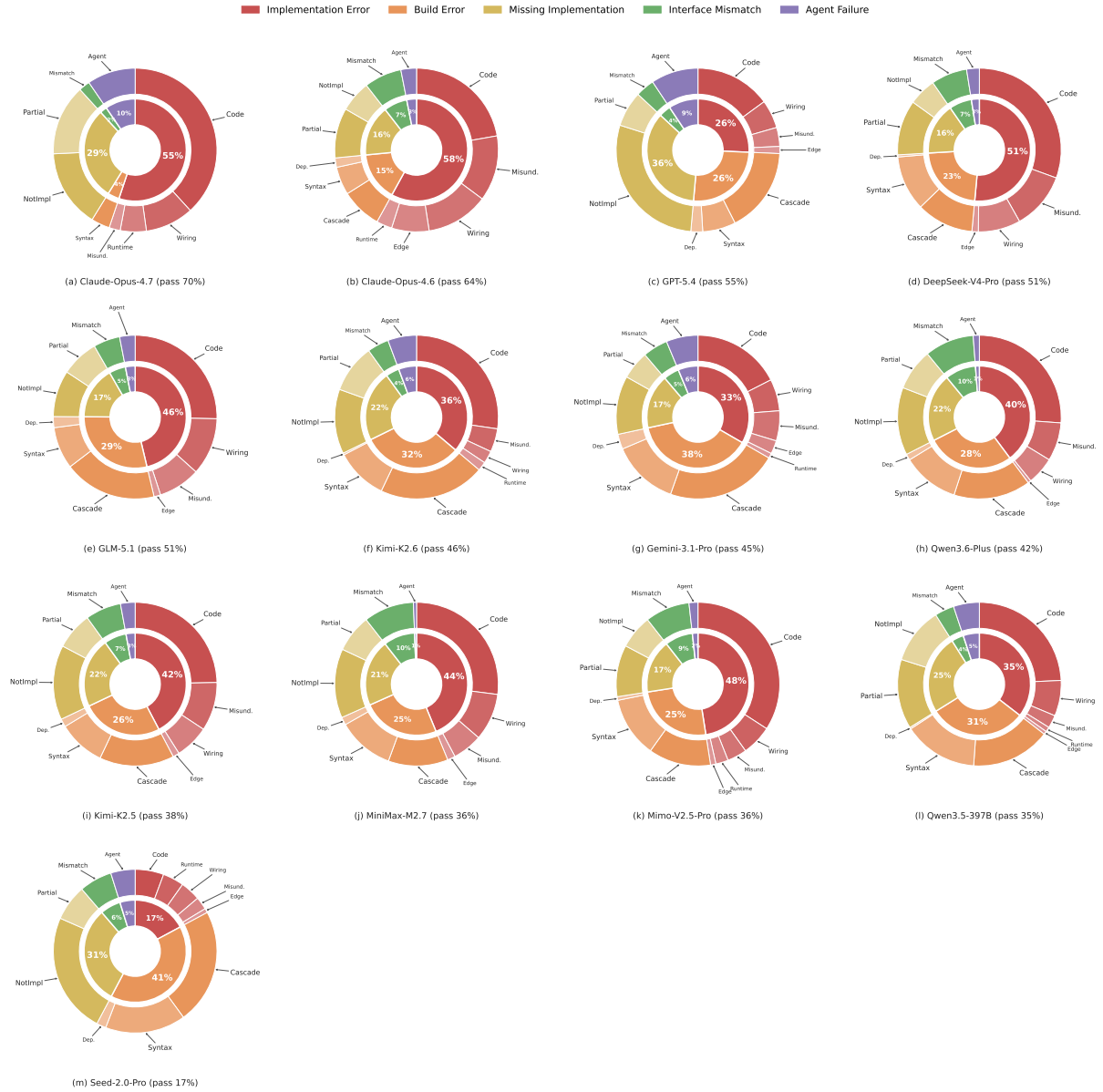


Figure 15: **Error distribution for all thirteen analyzed models** (inner ring: category proportions; outer ring: sub-type breakdown). Models are ordered by decreasing subtask pass rate from (a) to (m). The dominant failure mode shifts from Implementation Error (strong models) to Build Error and Missing Implementation (weak models).

MiniMax-M2.7 (pass 36%). Highest Implementation Error percentage among mid-tier models (44%), with Interface Mismatch also elevated (10%). Agent Failure is nearly zero (1%), meaning the model always attempts implementation—but frequently produces incorrect results.

Mimo-V2.5-Pro (pass 36%). Implementation Error dominates (48%), with Code Defect at 34%—the highest raw Code Defect rate among all models. Interface Mismatch is elevated (9%), split between Wrong Signature (15) and Wrong Path (10). Partially Implemented (29) exceeds Not Implemented (19), indicating the model attempts most features but often delivers incomplete solutions.

Qwen3.5-397B (pass 35%). Balanced across Implementation Error (35%), Build Error (31%), and Missing Implementation (25%). Syntax Error is notably high within Build Error (46 of 96), suggesting frequent compilation-level mistakes rather than cascading propagation. Partially Implemented (43) strongly

dominates Not Implemented (35), a pattern distinct from weaker models where Not Implemented typically leads.

Seed-2.0-Pro (pass 17%). Dominated by Build Error (41%) and Missing Implementation (31%). Implementation Error accounts for only 17%—not because the model is precise, but because code often fails to compile before behavioral correctness can be evaluated. This model represents the weakest capability tier where fundamental code generation is the bottleneck.

H.5 Representative Case Studies

We present one representative case per error category, selected to demonstrate how each failure type manifests in practice. Each case includes the target requirement, the key test output, and root-cause analysis.

Case 1: Implementation Error (Code Defect)

pyg-1.7.2-roadmap Target 6: Regularization Functions | Kimi-K2.5

Target Requirement: Implement a `gini(w: Tensor)` function that computes the Gini coefficient of a 2D weight matrix (row-wise inequality of absolute values, averaged across rows). A fully sparse row gives Gini close to 1.0; uniform gives 0.0. For a matrix with row 1 = $[0, 0, 0, 0]$ and row 2 = $[0, 0, 0, 1000]$, the expected Gini is 0.5.

Test Output (1 failed / 7 total):

```
FAILED test_gini_regularization
  assert torch.isclose(result, torch.tensor(0.5))
  AssertionError:
    tensor(0.1250) != tensor(0.5000)
```

Root Cause: The normalization formula is inverted. The agent wrote $\text{gini} = \text{gini} / (n - 1)$ instead of the correct $\text{gini} = \text{gini} * n / (n - 1)$. For the test input $[0, 0, 0, 1000]$: raw Gini = 0.75, correct normalized = $0.75 \times \frac{4}{3} = 1.0$, but the implementation computes $0.75 \div 3 = 0.25$. Averaging with the all-zero row (Gini = 0) yields 0.125 instead of the expected 0.5.

Insight: Six of seven tests pass—the function exists, compiles, and handles most cases correctly. The failure is a single arithmetic operator error ($/$ vs. $*$) in a normalization formula, exemplifying the “execution precision” bottleneck: models understand the algorithm but make subtle mistakes when translating mathematical specifications to code.

Case 2: Build Error (Circular Import)

fal-1.3.0-roadmap Target 1: Media Framework | GPT-5.4

Target Requirement: Create a pluggable media handling system: `BaseHandler` abstract class, `Handlers` registry mapping content types to handler instances, `JSONHandler/MessagePackHandler` implementations, and a `validate(schema)` decorator for JSON Schema validation. Add media properties on `Request/Response` for automatic serialization.

Test Output (0 collected, import error):

```
ERROR collecting test_01_media.py
  ImportError while importing test module:
    falcon/__init__.py:32
      -> falcon/api.py:21
      -> falcon/routing/__init__.py:22
      -> falcon/routing/compiled.py:21:
          import falcon.routing.converters
  AttributeError: module 'falcon' has no attribute 'routing'
```

Root Cause: Agent used absolute `import import falcon.routing.converters` in `compiled.py`. This statement requires Python to resolve `falcon.routing` as an attribute of the `falcon` module object. However, at this point in the initialization sequence (`falcon.__init__` → `falcon.api` → `falcon.routing.__init__` → `compiled.py`), the `falcon.__init__` module has not finished executing, so the `routing` attribute has not yet been bound to the `falcon` module namespace—even though `falcon/routing/__init__.py` is actively being loaded. The fix is to use a relative import (`from . import converters`). The failure cascades to all 5 subtasks (no tests can be

collected).

Insight: A single import-path mistake renders the entire codebase unimportable, demonstrating how Build Errors—especially cascading ones—produce catastrophic multi-target failures.

Case 3: Missing Implementation (Not Implemented)

opt-3.2.0-roadmap Target 4: BIPOP CMA-ES | Seed-2.0-Pro

Target Requirement: Extend `CmaEsSampler` to accept `restart_strategy="bipop"`, implementing BI-population CMA-ES that alternates between large-population and small-population restarts. Add `n_restarts_with_large`, `poptype`, `small_n_eval`, `large_n_eval` fields to the `_CmaEsAttrKeys` `NamedTuple`. Invalid strategy values must raise `ValueError`.

Test Output (16 failed / 17 total):

```
FAILED test_bipop_available
  ValueError: restart_strategy=bipop is unsupported.
  Please specify: 'ipop', 'bipop' or None.
```

```
FAILED test_sampler_attr_key_bipop[options0-cma:]
  AttributeError: '_CmaEsAttrKeys' object has no
  attribute 'n_restarts_with_large'
```

```
FAILED test_restore_optimizer_after_restart_bipop
  ValueError: restart_strategy=bipop is unsupported.
```

Root Cause: Agent left only # `TODO(c-bata): Support BIPOP-CMA-ES. without implementing any functionality.` The `restart_strategy` validator still only accepts `'ipop'` and `None`; the `_CmaEsAttrKeys` `NamedTuple` was never extended. The sole passing test (`test_invalid_restart_strategy`) checks that truly invalid values (e.g., `'foo'`) raise exceptions—it passes because `'bipop'` is now also rejected, though it should have been a valid option.

Insight: Weak models skip complex algorithmic requirements entirely rather than attempting partial implementations, resulting in a pattern of `TODO` comments as placeholders.

Case 4: Interface Mismatch (Wrong Export Path)

fal-3.0.0-roadmap Target 1: ASGI Support | Qwen3.6-Plus

Target Requirement: Create `falcon.asgi` package with `async App`, `Request`, `Response`, `BoundedStream`. Implement testing utilities (`ASGIConductor`, `create_scope()`, `SimpleTestResourceAsync`) and `sync/async` bridge functions (`sync_to_async`, `async_to_sync`). All must be importable from `falcon.testing` and the top-level `falcon` namespace.

Test Output (12 failed / 43 total):

```
FAILED test_asgi_conductor_default_headers
  ImportError: cannot import name 'ASGIConductor'
  from 'falcon.testing'
```

```
FAILED test_sync_to_async
  falcon/util/sync.py:21:
  RuntimeError: no running event loop
```

```
FAILED test_unsupported_http_version[0.9]
  Failed: DID NOT RAISE UnsupportedError
```

Root Cause: Agent implemented `ASGIConductor`, `SimpleTestResourceAsync`, and `create_scope` in `testing/asgi_client.py`, but `testing/__init__.py` never imports from that file. These classes exist on disk but are inaccessible via the `falcon.testing` namespace that tests use. Additionally, `async_to_sync` calls `asyncio.get_event_loop()` which fails on Python 3.10+ without a running loop.

Insight: Writing correct code is necessary but not sufficient—the code must also be properly exported at the expected module path. This class of error is especially common in Python packages with explicit `__init__.py` re-exports.

Case 5: Agent Failure (Infinite Loop Until Budget Exhaustion)

mko-4.0.0-roadmap Target 1-7: ORM Core Decorators | Claude-Opus-4.6

Target Requirement: Implement seven core ORM decorators (@Filter, @Subscriber, @Embeddable, @Formula, etc.) with full TypeScript decorator semantics, metadata storage, and integration with the entity manager lifecycle.

Agent Trajectory (249 steps, infinite loop):

Step 1-40: monorepo directory restructuring
Step 41-180: repeated attempts to reorganize packages/
Step 181-245: trapped in a loop:
Step 245: ls packages/core/src/ | wc -l
Step 246: ls packages/core/src/ | wc -l
Step 247: ls packages/core/src/ | wc -l
...
Step 249: (budget exhausted, forced termination)
=== zero decorator implementations written ===

Test Output (all 7 targets fail identically):

TypeError: core_1.Filter is not a function
TypeError: core_1.Subscriber is not a function
TypeError: core_1.Embeddable is not a function
(all decorators undefined - never implemented)

Root Cause: The agent spent its entire 249-step budget on monorepo directory restructuring (copying files between directories, checking file counts) without ever beginning to implement any decorator. In the final 60+ steps, the agent entered a degenerate loop, repeatedly executing the same `ls | wc -l` command with no progress. The session was forcibly terminated at the step limit.

Insight: This is the canonical Agent Failure pattern: the agent gets stuck in preparatory work and never reaches the actual implementation. Unlike Missing Implementation (Case 3) where a feature is consciously skipped, here the agent *intended* to implement but was trapped in an unproductive loop until its budget was exhausted.